

CS2300: File Structures and Introduction to Database Systems

*Conceptual Data Modeling and Database
Design*

Lecture 3:
Data Modeling Using the ER Model

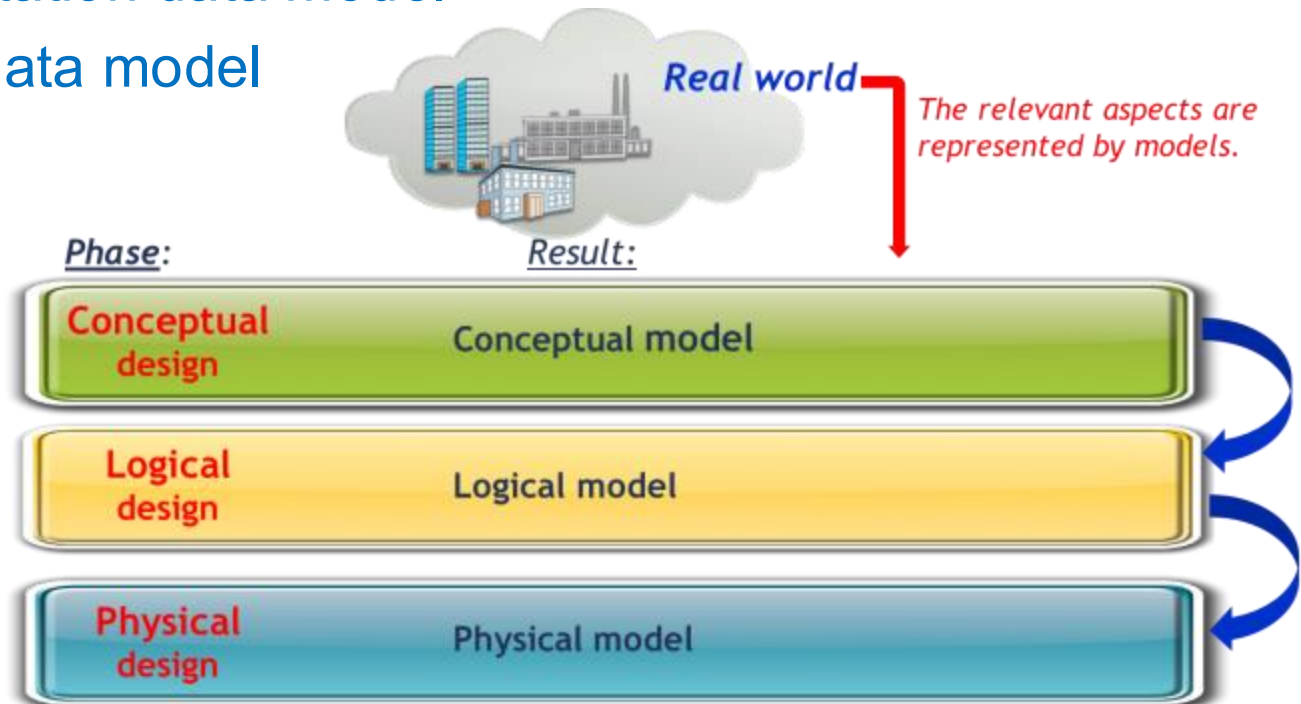
San Yeung

How a Database is Created

- Databases are a crucial component in the information system within any organization
- The design of a brand-new database starts off with the ***requirements specification and analysis*** phase
 - **Conceptual Design:**
 - Represent database requirements using some computerized tools
 - E.g. **Entity-Relationship Model**
 - **Logical Design:**
 - The translation of the conceptual design into a data model implemented in a commercial DBMS
 - **Physical Design:**
 - Executing implementational decisions for storing and accessing the database

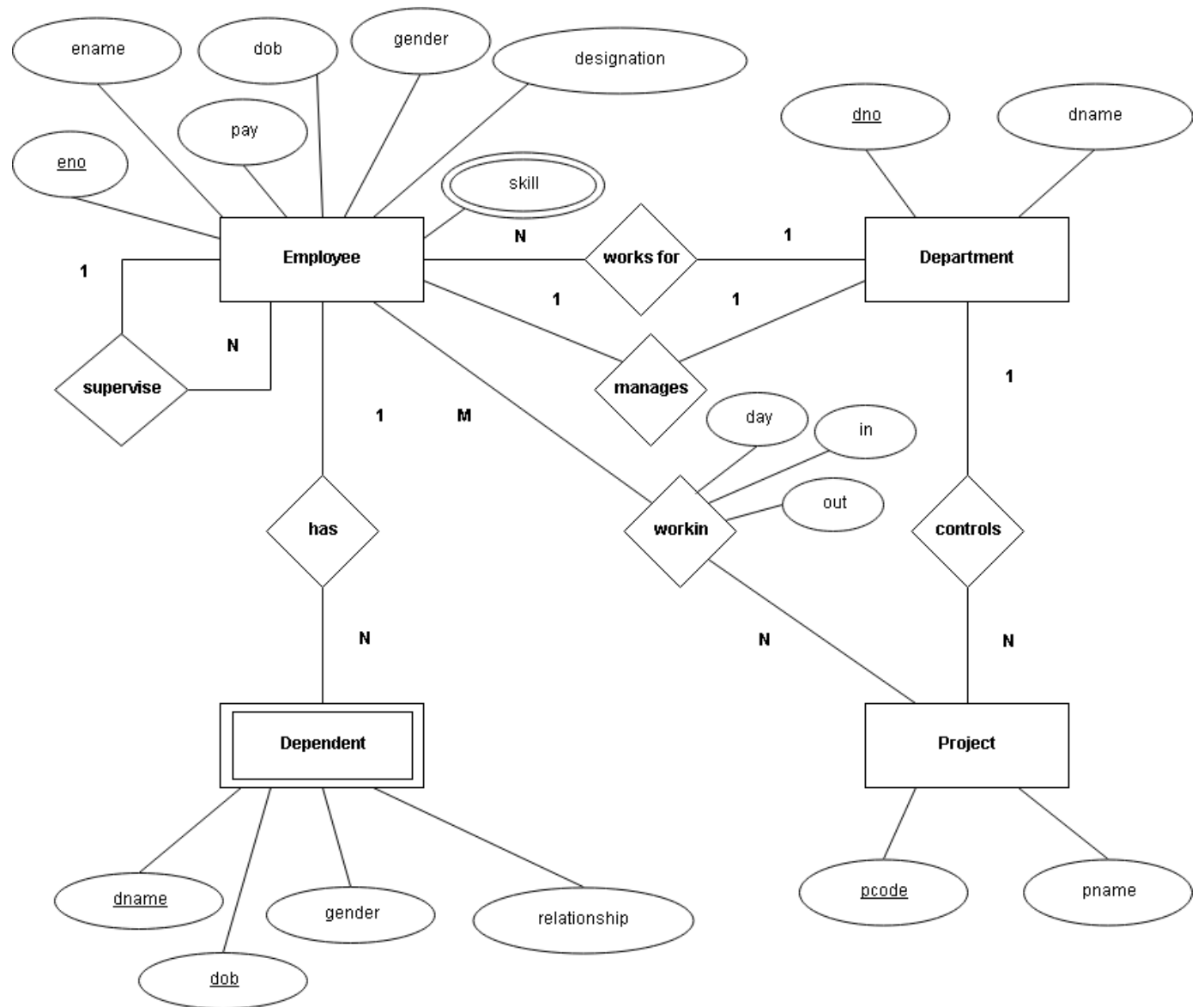
What is a Data Model?

- Categories of data models
 - Conceptual data model
 - Implementation data model
 - Physical data model



What is a Data Model?

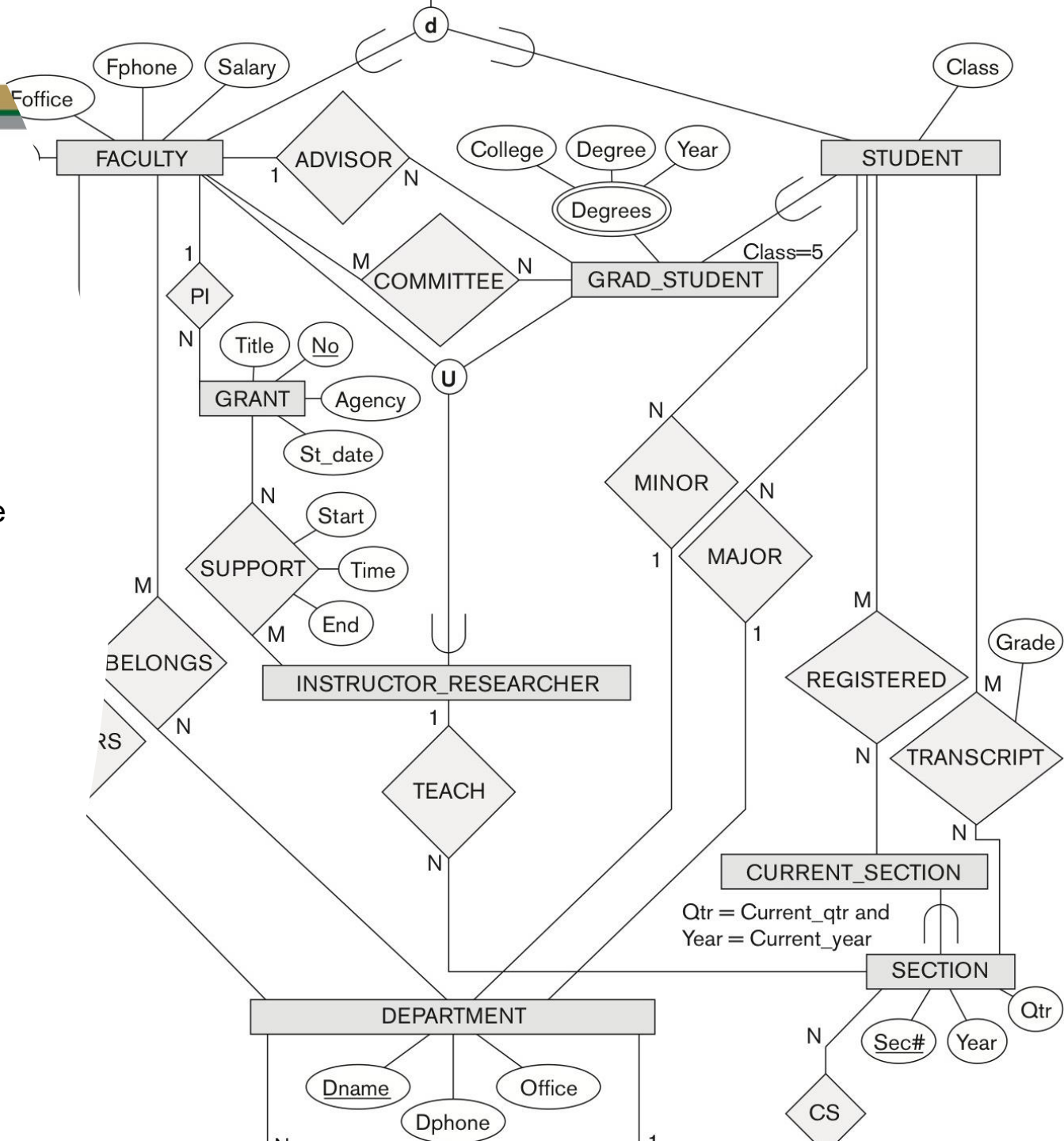
- An *abstract* representation
- A *structured framework* for conceptualizing, organizing, storing, and manipulating data
 - Specifies the nature of data
 - Relationships among data entities
 - Constraints on the data
- Hides low level details



Conceptual Data Models

- *High level*: from the user's perspective
- For end user/customer (non-technical stakeholders)
- Does not care about low-level details
 - How to store data?
 - Which type of DBMS to use?
- For example: *E-R diagrams*
 - *Entities*: represent a realistic object or concept
 - *Attributes*: some properties that describe the entity
 - *Relationships*: an association among two or more entities

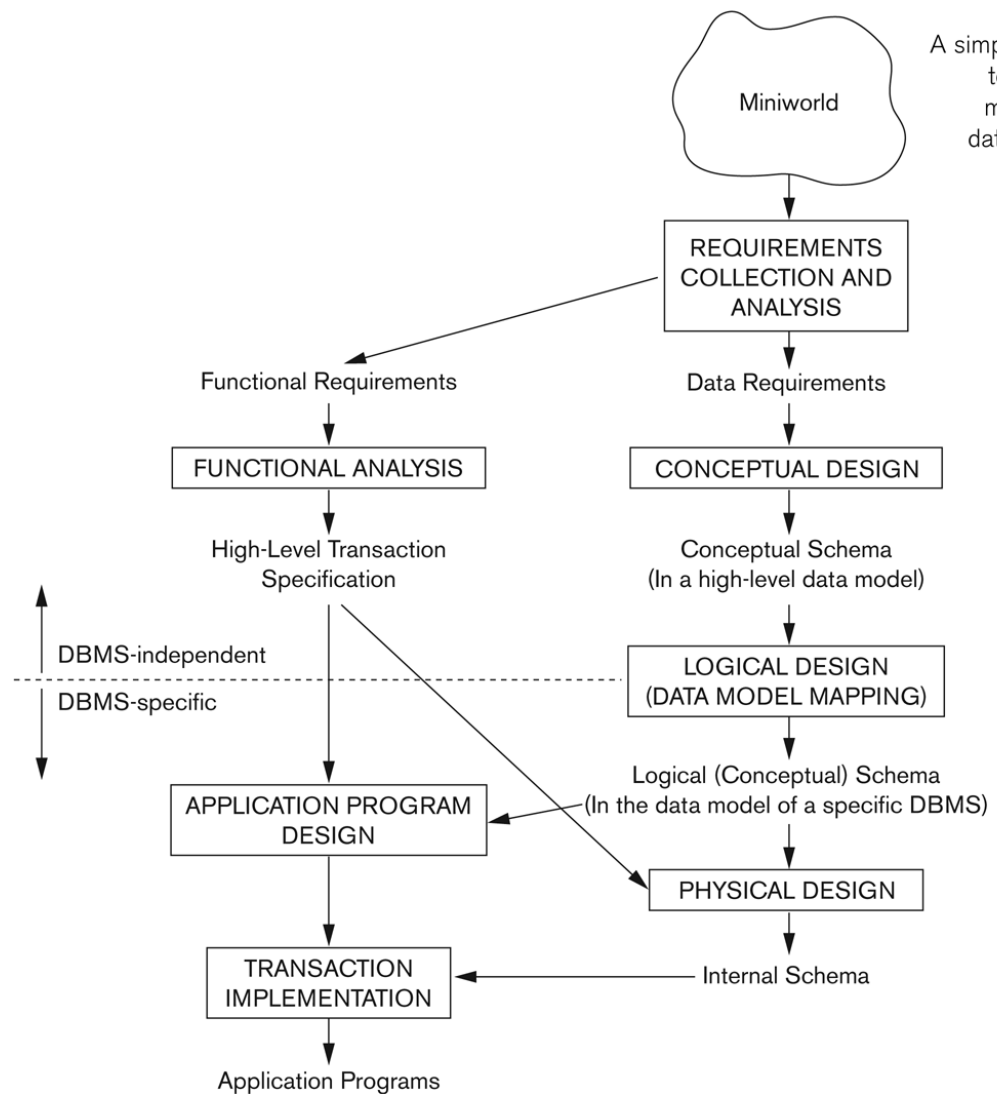
- An ER diagram example of a UNIVERSITY database
- Used during the initial design phase to define the high-level structure and relationships of the database



Introduction

- **Conceptual modeling** is a very important phase in designing a successful database application
- **database application:**
 - mainly consists of a particular database
 - and the associated programs implemented for database queries and updates
- A major part of the database application will require the *design*, *implementation*, and *testing* of the application programs.
- In many software design tools, database design and software engineering methodologies are often intertwined

Using High-Level Conceptual Data Models for Database Design



Using High-Level Conceptual Data Models for Database Design

1. Requirements collection and analysis

1. The database designers must first understand and document the **data requirements** of the users
2. Useful to specify also the known **functional requirements** of the application
 1. E.g. *user-defined operations* for both retrievals and updates

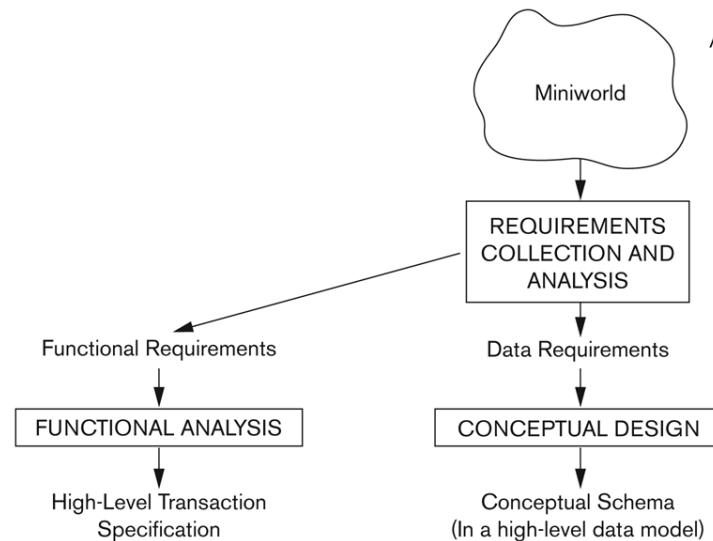


Figure 3.1

A simplified diagram to illustrate the main phases of database design.

Using High-Level Conceptual Data Models for Database Design

2. Conceptual design

1. Use a high-level conceptual data model to create a **conceptual schema** for the database
2. The schema needs to be easy to understand and communicate with nontechnical users
3. Could be used as a reference to ensure that all users' data requirements are met and conflict-free

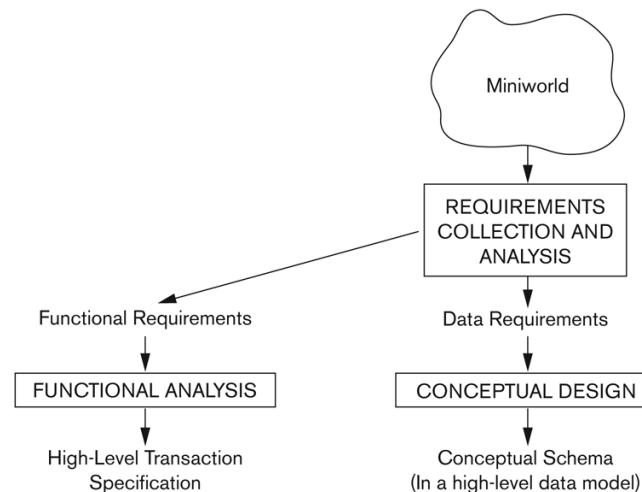


Figure 3.1

A simplified diagram to illustrate the main phases of database design.

A Sample Database Application

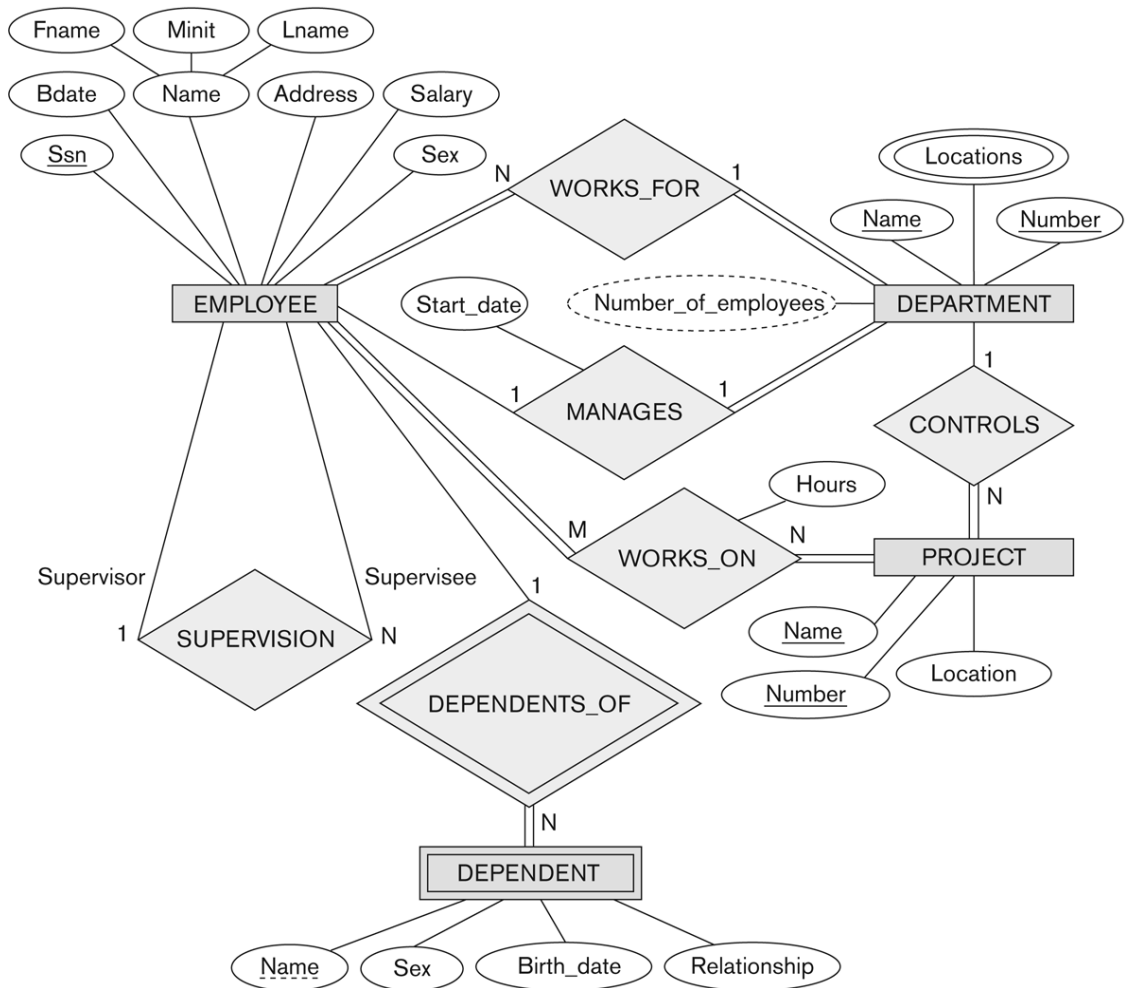


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.



A Sample Database Application

- A COMPANY database to keep track of the company's employees, departments, and projects
- COMPANY *database schema*:
 - The company is organized into **departments** and each department may have several locations
 - A department controls a numbers of **projects**
 - An **employee** is assigned to one department but may work on several projects
 - The database should keep track of the **dependents** and the direct **supervisor** of each employee
- Figure 3.2 displays the schema using the graphical notation known as **ER diagrams**



Entity Types, Entity Sets, Attributes, and Keys

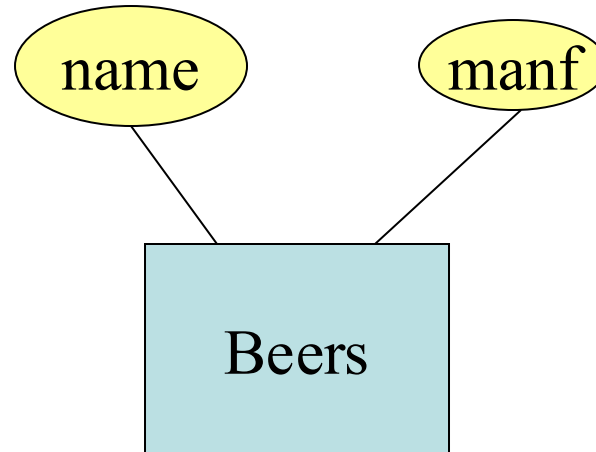
- The **Entity-Relational (ER) model** allows us to sketch the design of a database informally
- Designs are pictures called entity-relationship diagrams
- The ER model describes data as *entities*, *relationships*, and *attributes*

Entities and Attributes

- **Entity** = a “thing” or object with an *independence existence*
 - **Physical existence:**
 - a particular person, car, house, or employee
 - **Conceptual existence:**
 - a company, a job, or a university course
- Each entity has **attributes**
 - Particular properties with descriptive values
 - E.g. an **EMPLOYEE** entity may be described by the employee’s ?_____, age, address, salary, and job
 - A specific entity will have a value for each of its attributes
 - The attribute values that describe each entity become a *major part* of the data stored in the database

ER Diagrams

- In an entity-relationship diagram, each entity set is represented by a **rectangle**.
- Each attribute of an entity set is represented by an **oval**, with a line to the rectangle representing its entity set.
- Example



ER Diagram

- Example: A student has his ID, name, phone, height, etc.

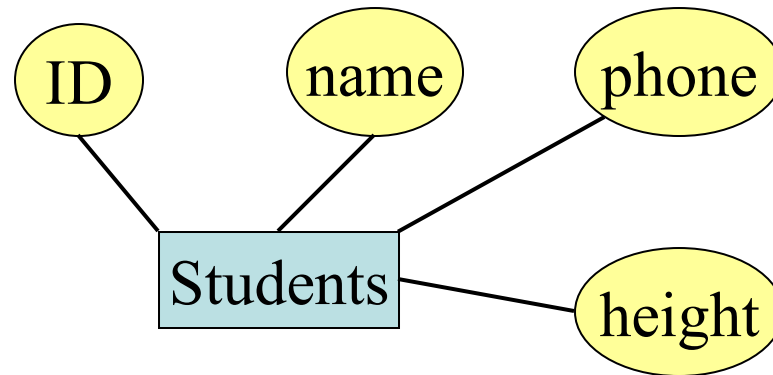
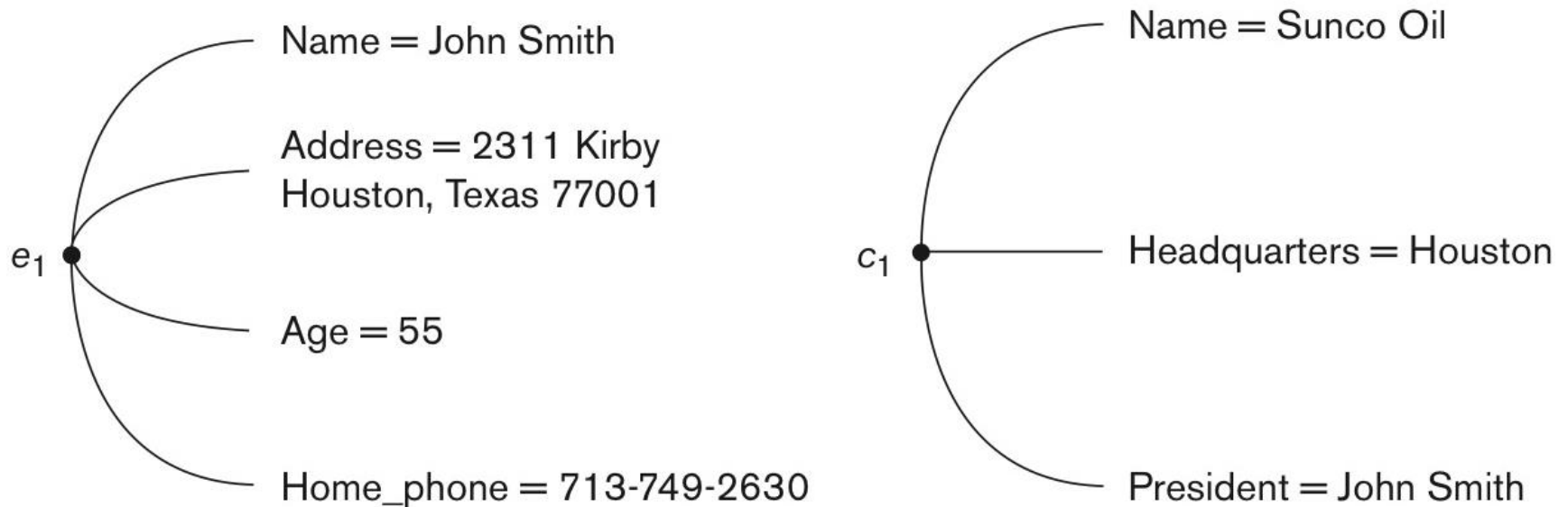


Figure 3.3 Two entities, EMPLOYEE e_1 , and COMPANY c_1 , and their attributes.



Types of Attributes

- *simple* vs *composite*
- *single-valued* vs *multivalued*
- *stored* vs *derived*

Question

If a phone number (e.g. 212-900-0102) is to be represented as an attribute for a database entity, then it is better to divide it into multiple components: an area code (212), an exchange number(900), and a four-digit code (0102).

- A) True, if there's a need for finer granularity.
- B) False, if the system prioritizes simplicity in data entry and retrieval.
- C) It depends on the specific requirements of the application or system, like intended usage, operations to be performed, etc.
- D) None of the above

Composite vs Simple (Atomic) Attributes

- **Simple**
 - Attributes that are **not divisible**
 - Example: ID, Gender.
- **Composite**
 - The attribute may be **composed of several components**.
 - Could be divided into smaller *subparts*
 - Example:
 - Name(FName, MName, LName)
 - Address(Street_address, City, State, Zip, Country)

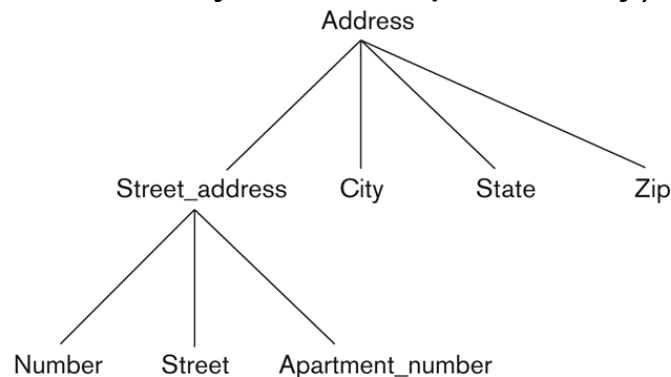
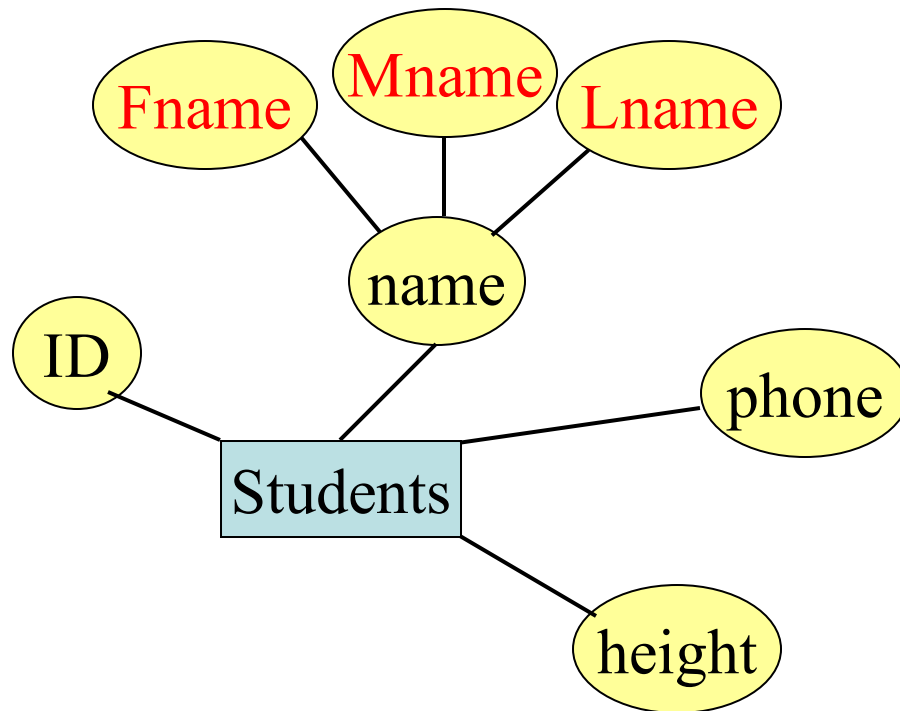


Figure 3.4
A hierarchy of
composite attributes.

Example of Composite Attributes



- Why would composite attributes be *useful*?
- When should we choose to use the *simple form*?

Single-Valued vs Multivalued Attributes

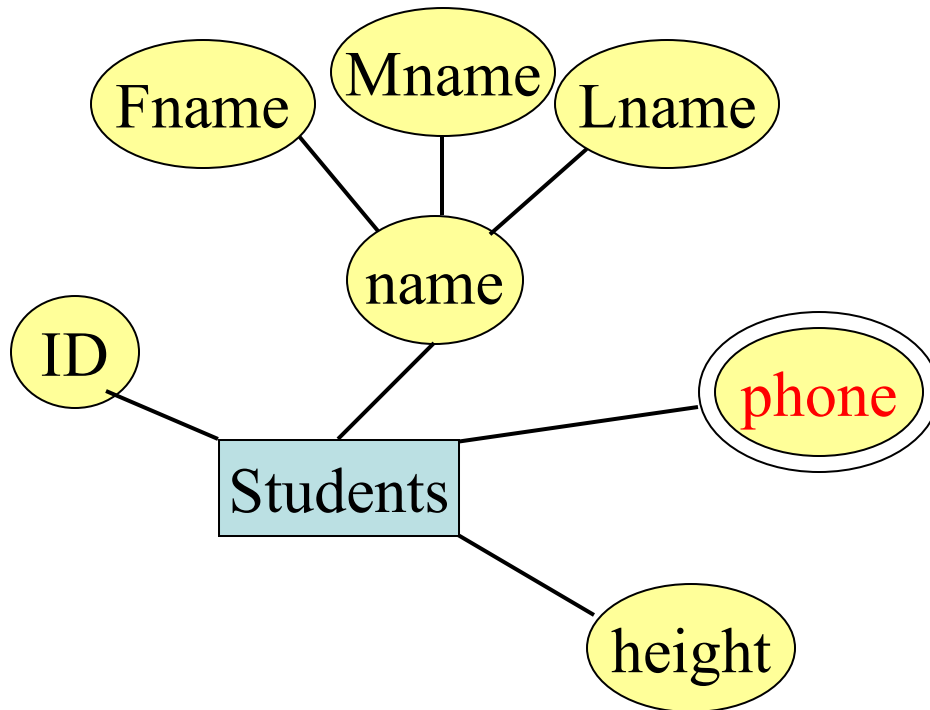
- **Single-Valued**

- Most attributes have a single value for a particular entity
- Example: age

- **Multivalued**

- An entity may have **multiple values** for the attribute
- Ex1) the “Colors” attribute for a car
 - A **two-tone** car could have two color values
- Ex2) the “College_degrees” attribute for a person
 - A person may not have any vs having two or more degrees

Example of Multivalued Attributes



Can “phone” be a multivalued attribute for the “Students” entity?

Would it be a violation if a student has 100 phone numbers?

- A multivalued attribute may have **lower** and **upper** bounds to **constrain** the number of values allowed for each individual entity

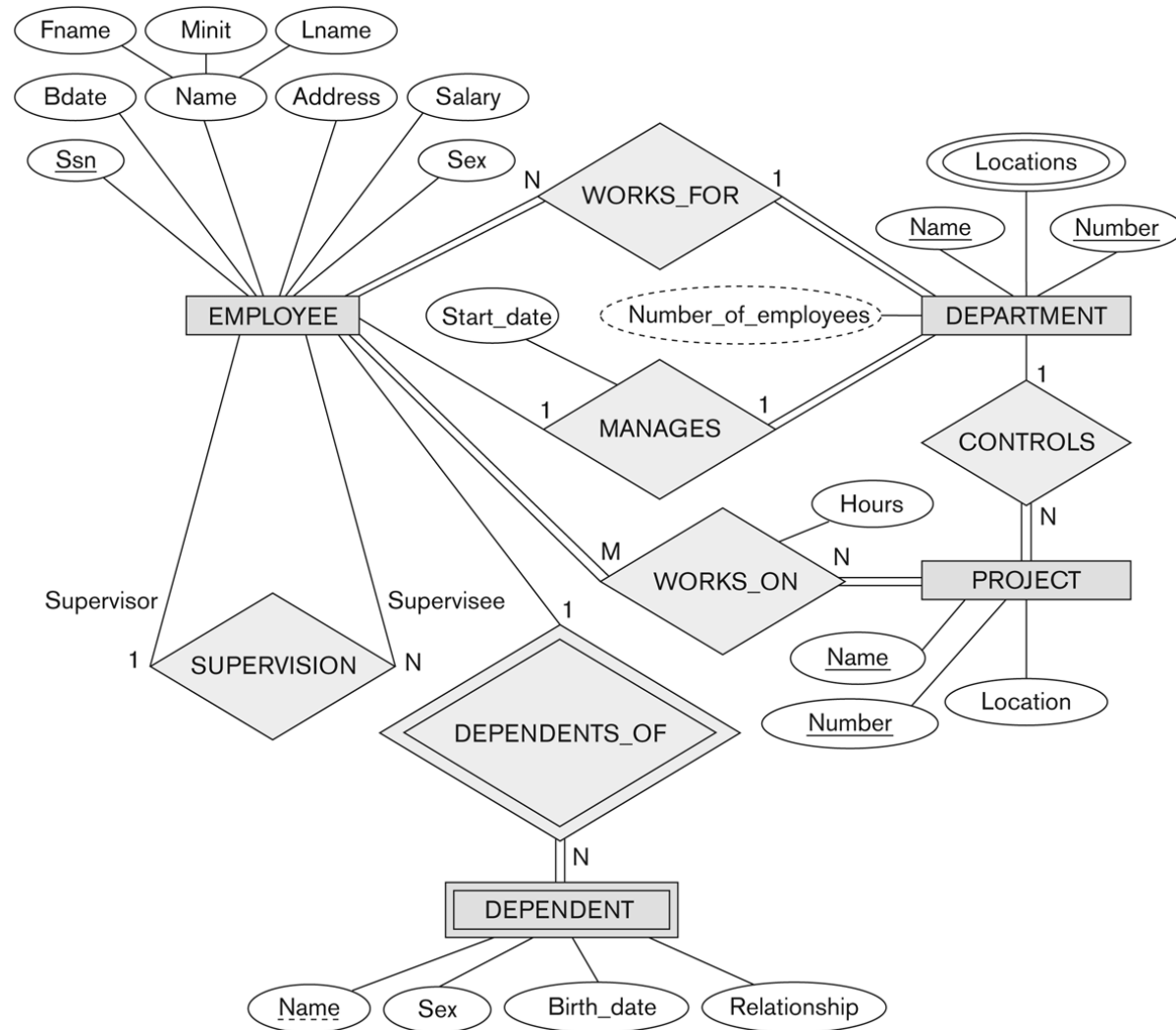


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Stored vs Derived Attributes

- **Stored** versus **Derived** attributes
 - One attribute is derivable from another attribute
 - Example: *age*, *birth_date*
 - The value of *age* can be determined from the value of *birth_date*.
 - *age* is a derived attribute
 - *birth_date* is a stored attribute
 - Some attributes can be derived from *related entities*
 - Example: an attribute “*number_of_employees*” of a Department entity

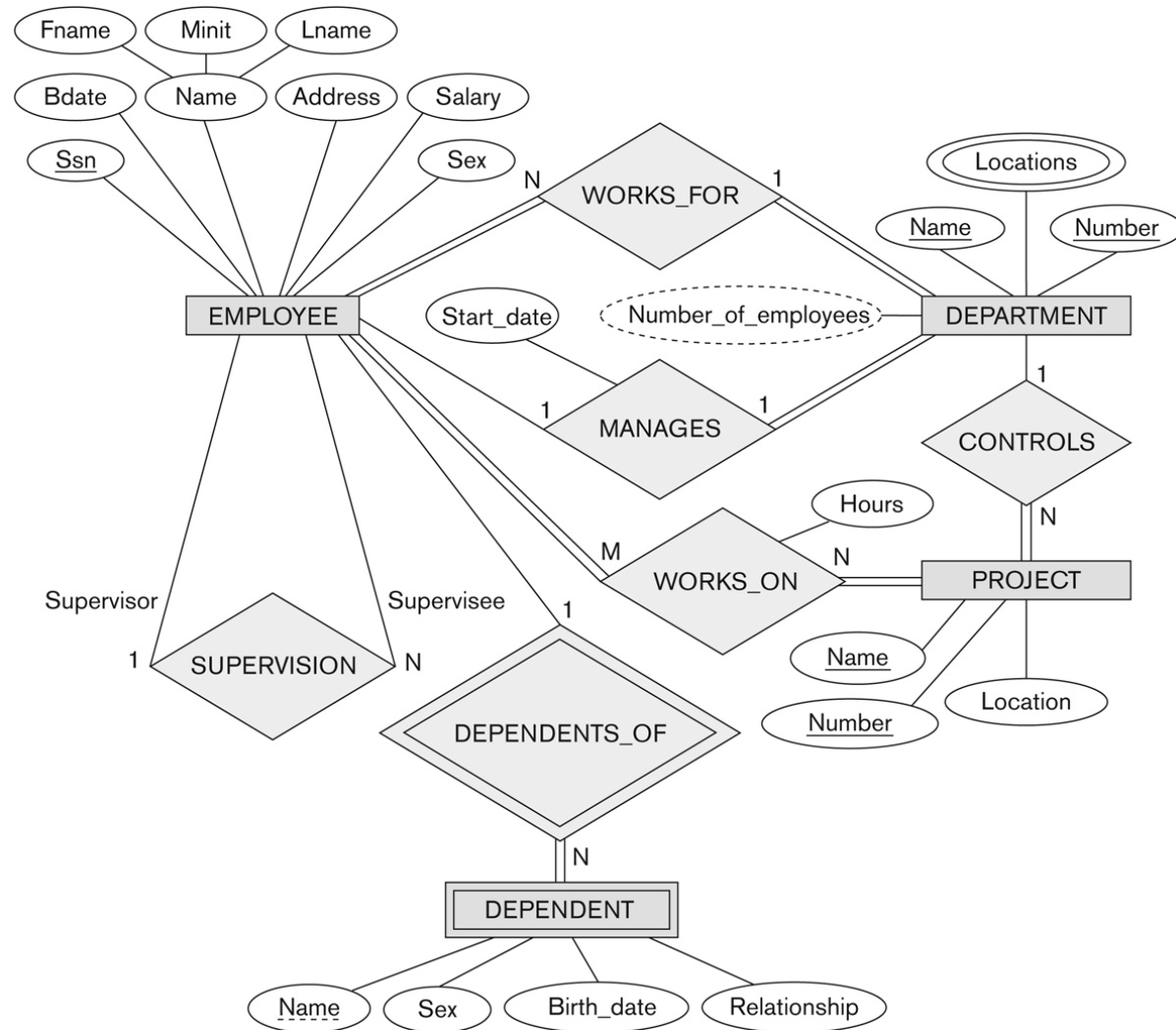


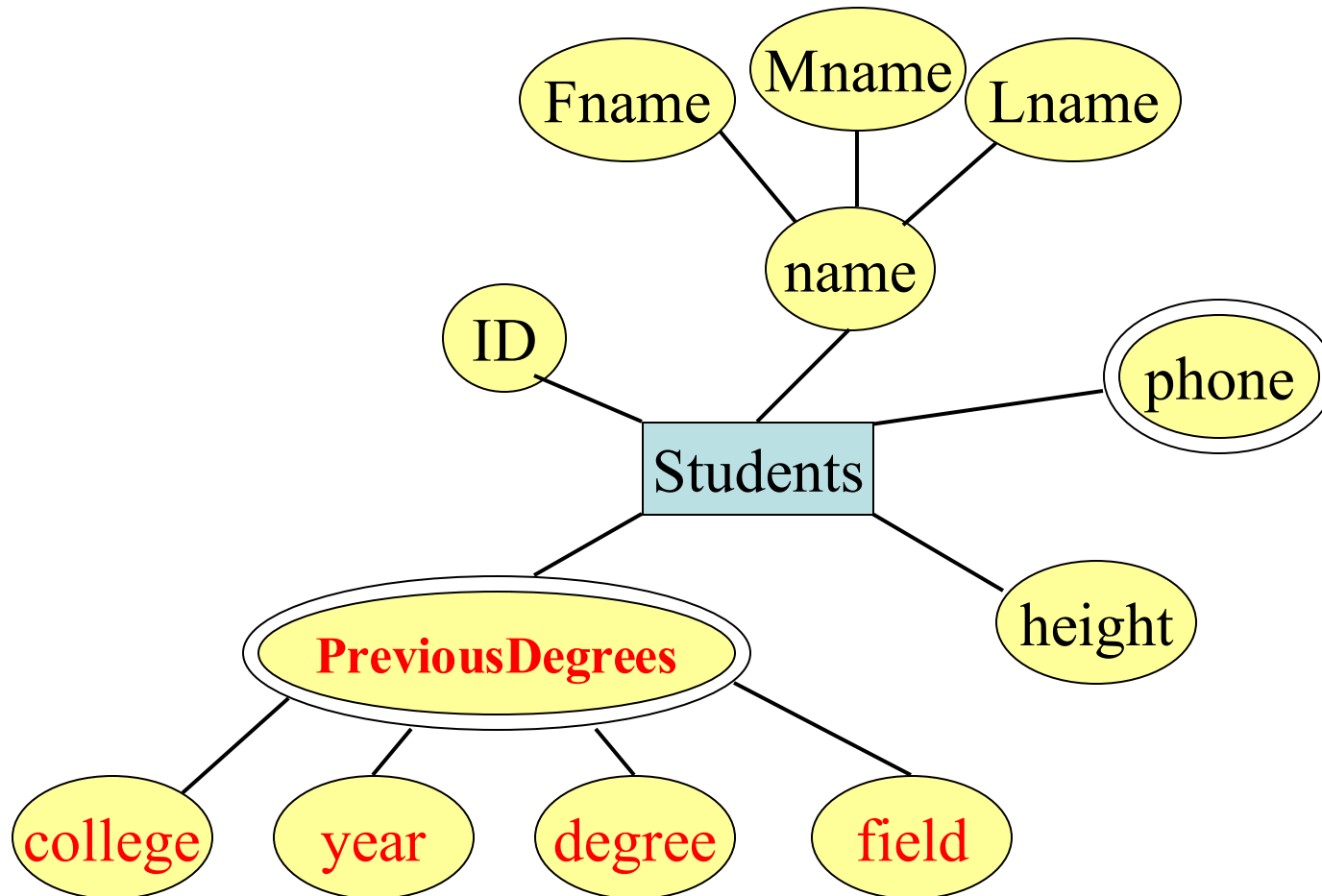
Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Complex Attributes

- **Complex Attributes**
 - Composite and multivalued attributes can be **nested** arbitrarily
 - For example, PreviousDegrees of a STUDENT is a composite multi-valued attribute
 - {PreviousDegrees (College, Year, Degree, Field)}
 - Each has four subcomponent attributes:
 - College, Year, Degree, Field
 - Multiple PreviousDegrees values can exist for a student

Example of Complex Attributes



Key Attributes of an Entity Type

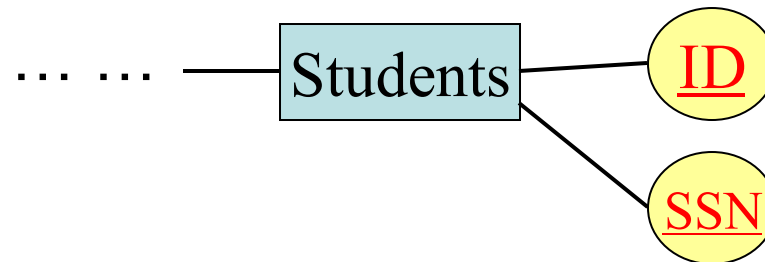
- **Uniqueness constraint (key):**

- an entity type usually has one or more attributes whose values are distinct for *each* entity in the entity set

- Ex)



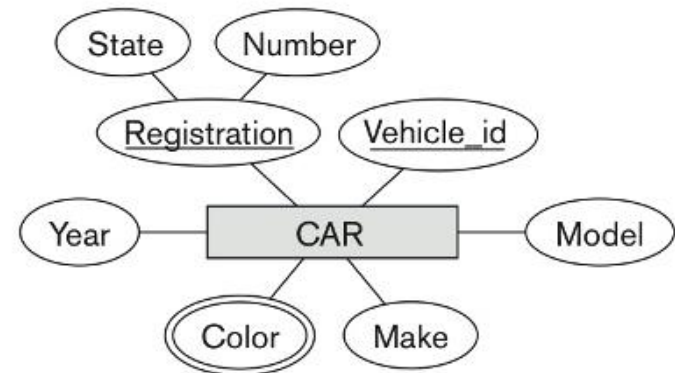
- Sometimes an entity type could have **more than one** key attributes



- Can an entity type have *no key*? If so, how do we uniquely identify each entity?

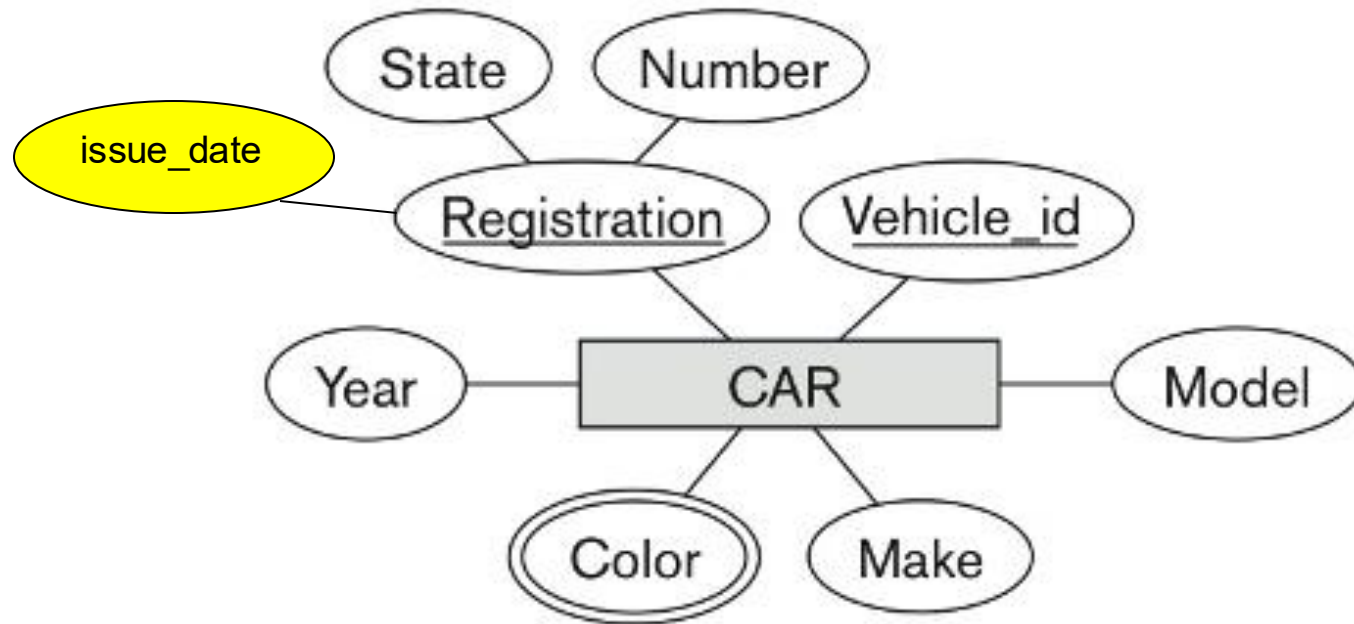
Key Attributes of an Entity Type

- Sometimes *several* attributes together form a key
 - → the **combination** of the attribute values must be distinct for each entity
 - The proper representation is to define a composite attribute and designate it as a key attribute
 - Such a composite key must be **minimal**



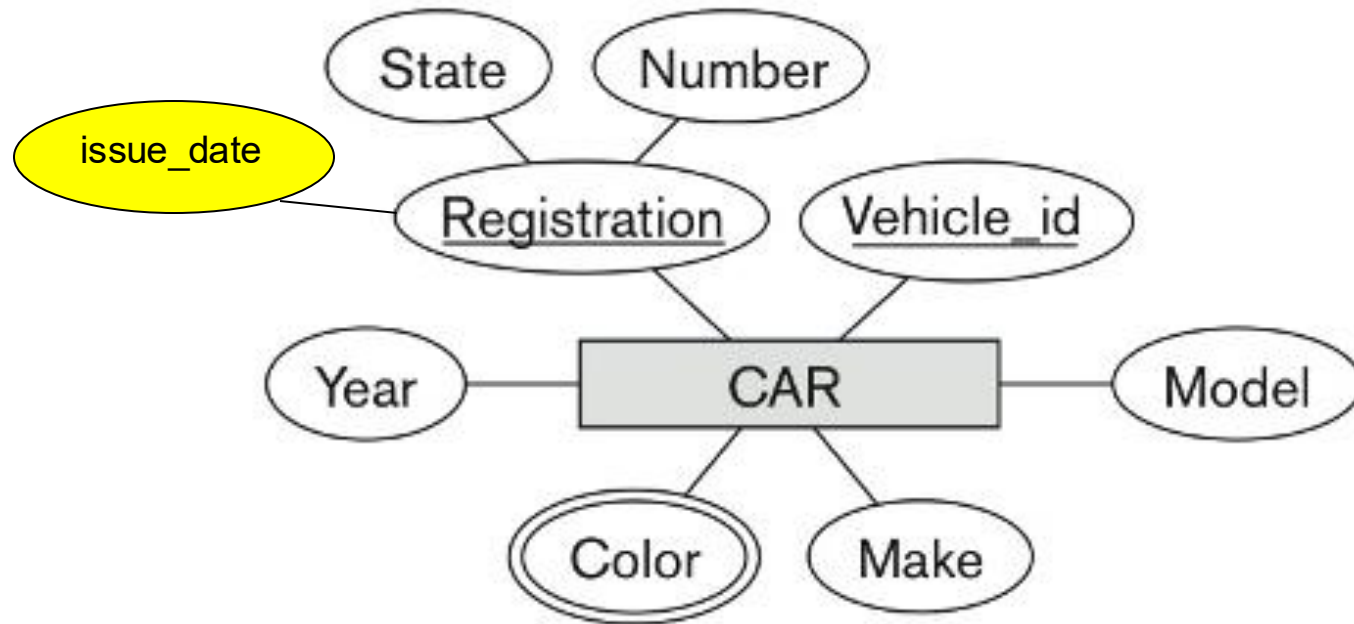
Key Attributes of an Entity Type

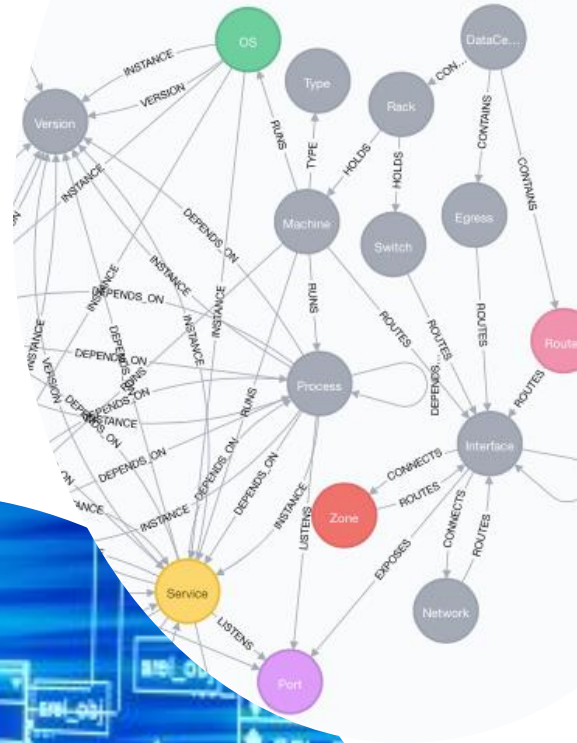
Minimal Composite Key?



Key Attributes of an Entity Type

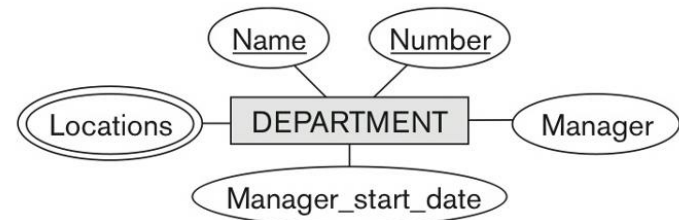
What's the correct notation?



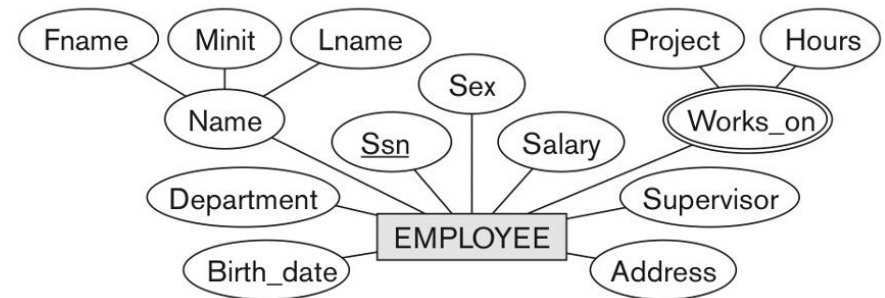


Relationship Types, Sets, Roles, and Structural Constraints

- Whenever an attribute of one entity refers to another entity type, some *relationship* exists
- In the ER model, these *references should not* be represented as attributes but as **relationships**
- Some attributes get converted into relationships between entity types as the initial design is refined



- The attribute *Manager* of DEPARTMENT refers to an employee who manages the department



- The attribute *Department* of EMPLOYEE refers to the department for which the employee works for

Relationship Types, Sets, and Instances

- A **relationship type** R among n entity types ($E_1, E_2, \dots, E_n$) defines a set of associations among entities from these entity types, called a **relationship set** (also referred to by R)
 - Each of the entity types is said to **participate** in the relationship type R
- A relationship set R is a set of **relationship instances** r_i , where each r_i associates n individual entities ($e_1, e_2, \dots, e_n$), and each entity e_j in r_i is a member of entity set E_j , $1 \leq j \leq n$.
 - Each of the individual entities ($e_1, e_2, \dots, e_n$) is said to **participate** in the relationship instance $r_i = (e_1, e_2, \dots, e_n)$.

Relationship type: **WORKS_FOR**

Participating entity types: 1) **EMPLOYEE** 2) **DEPARTMENT**

Relationship set: *all the works_for instances*

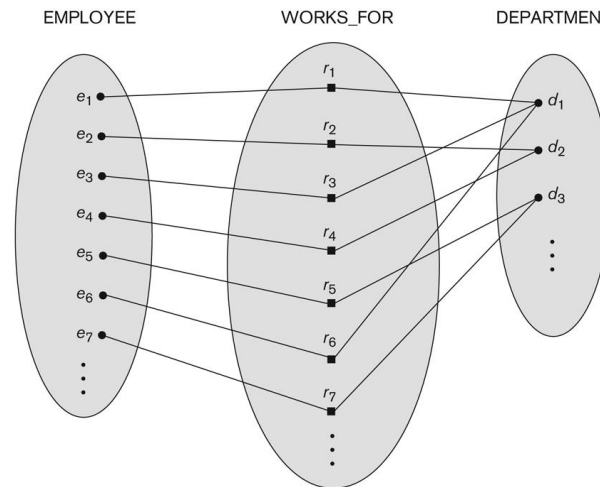
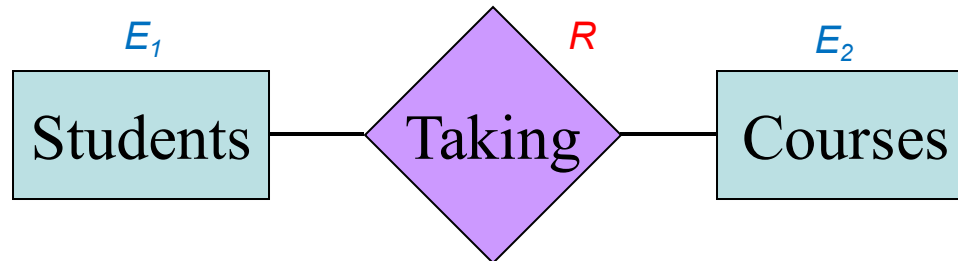


Figure 3.9

Some instances in the **WORKS_FOR** relationship set, which represents a relationship type **WORKS_FOR** between **EMPLOYEE** and **DEPARTMENT**.

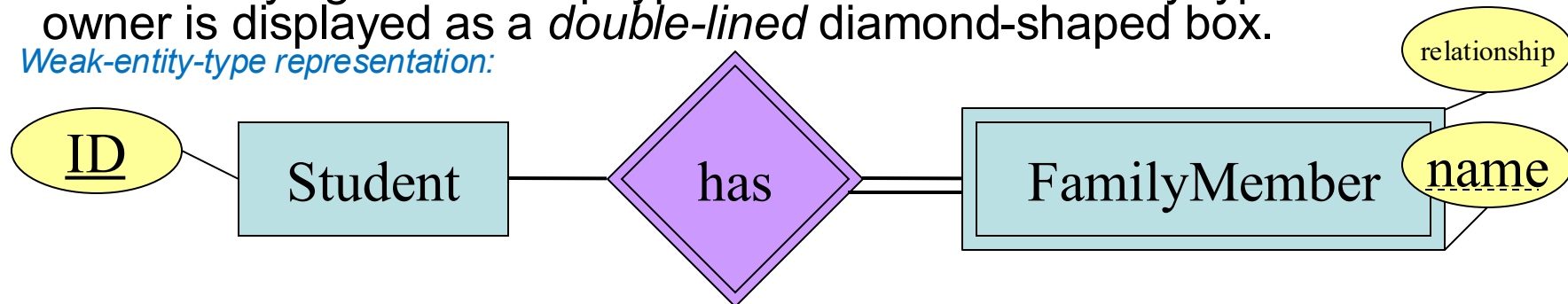
Relationship Types, Sets, and Instances

In ER diagrams, relationship types are displayed as *diamond-shaped boxes*, connected by straight lines to the participating entity types.



The identifying relationship type between a weak entity type and its owner is displayed as a *double-lined* diamond-shaped box.

Weak-entity-type representation:



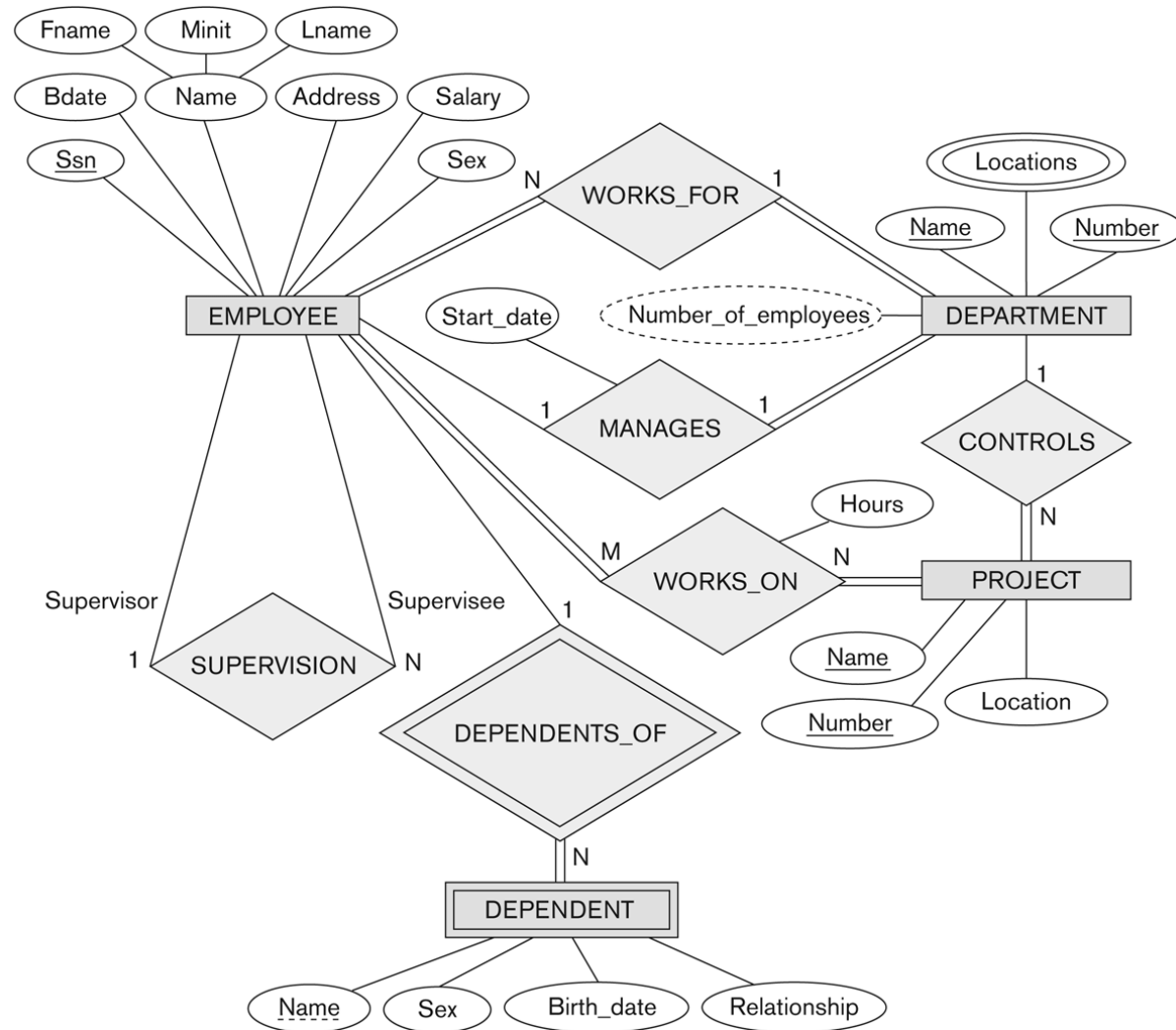


Figure 3.2

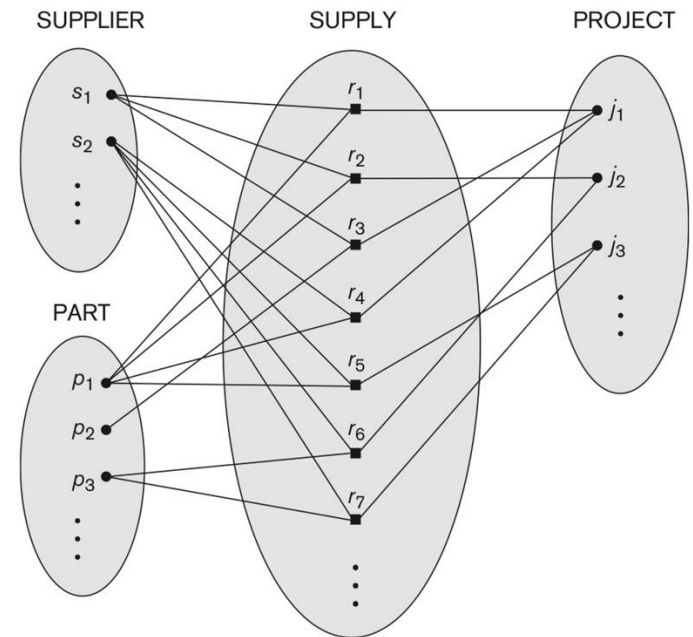
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Relationship Degree, Role Names, and Recursive Relationships

- **Degree of a relationship type:**
 - The **degree** of a relationship type is the number of participating entity types
 - A relationship type of degree two is called ?_____?
 - A relationship type of degree three is called ?_____?
- Relationships can generally be of *any* degree
- The most common ones are binary relationships
- Higher-degree relationships are generally more complex than binary relationships

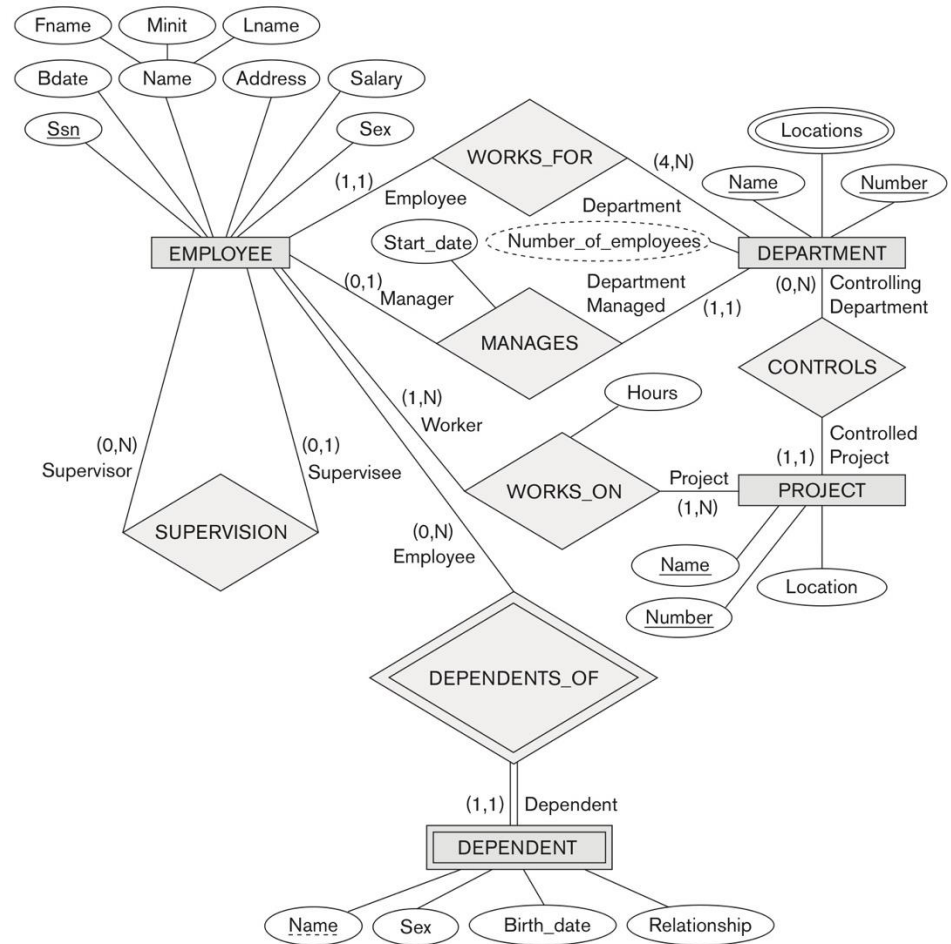
Relationship Degree, Role Names, and Recursive Relationships

- **An example of a ternary relationship: SUPPLY**
 - Each relationship instance associates three entities
 1. Supplier s
 2. Part p
 3. Project j
 - *What's the meaning of this relationship type?*
 - How can we make the meaning of a relationship more **self-explanatory**?



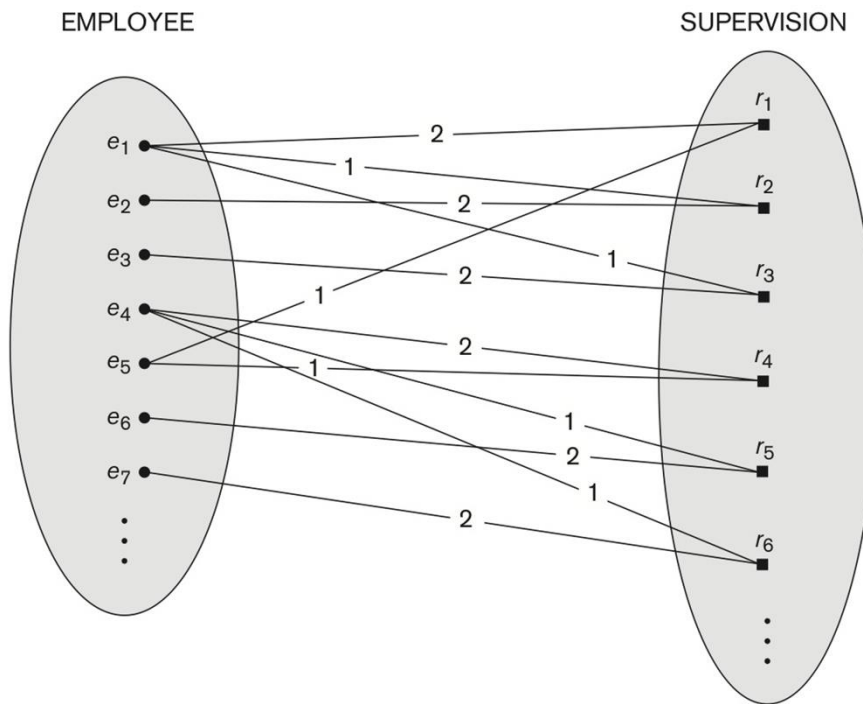
Role Names

- Each entity type that participates in a relationship type plays a particular **role** in the relationship
- Using *role names* help to explain what the relationship means
- For example:
 - The MANAGES relationship type
 - EMPLOYEE plays the role of *manager*
 - DEPARTMENT plays the role of *department to be managed*



Recursive Relationships

Figure 3.11 A recursive relationship SUPERVISION between EMPLOYEE in the *supervisor* role (1) and EMPLOYEE in the *subordinate* role (2).



- Role names are not technically necessary in all relationship types
- However, using role names becomes essential for *self-referencing relationships*:
 - the **same** entity type participates more than once in a relationship type in *different roles*
- An example for a recursive relationship: **SUPERVISION**

Recursive Relationships

- The SUPERVISION relationship type *relates* an *employee* to a *supervisor*
 - both entities are members of the **same** EMPLOYEE entity set
- The EMPLOYEE entity type participates *twice* in SUPERVISION
 - Once in the role of *supervisor*
 - Once in the role of *supervisee*

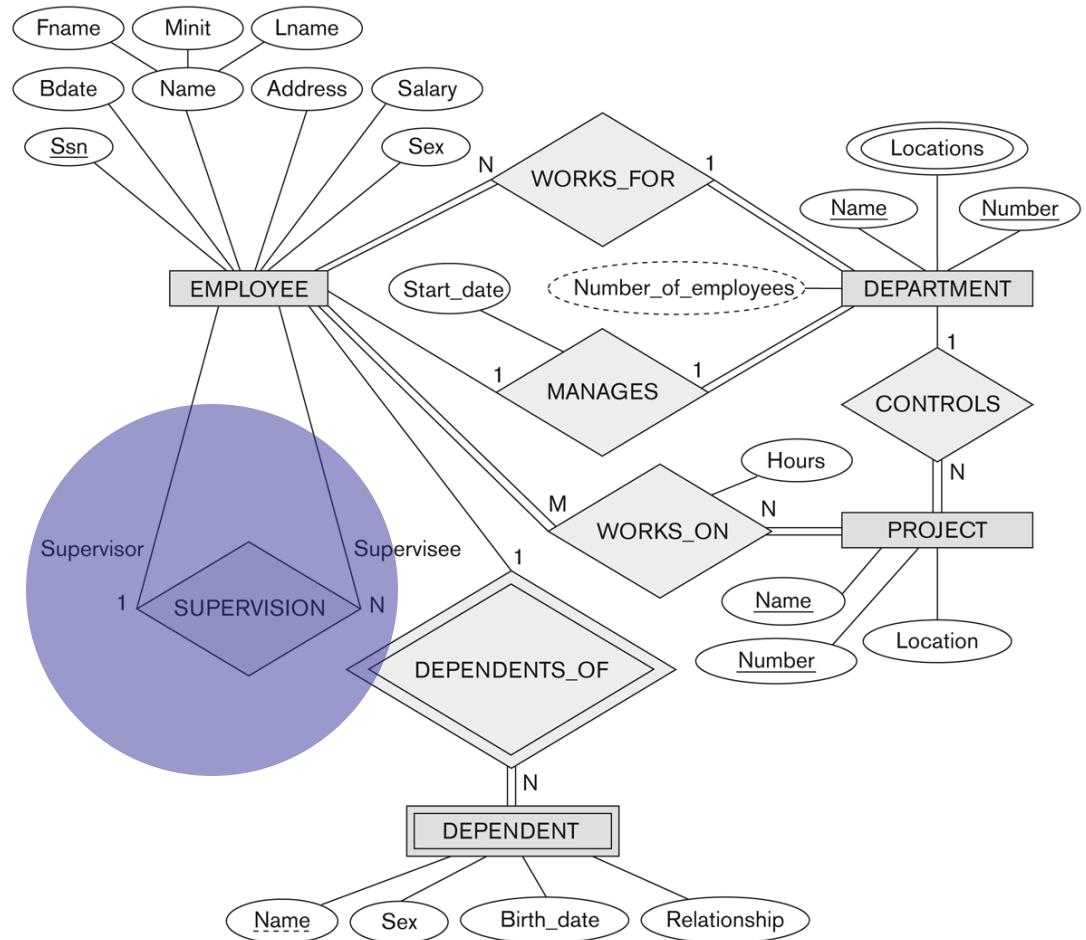
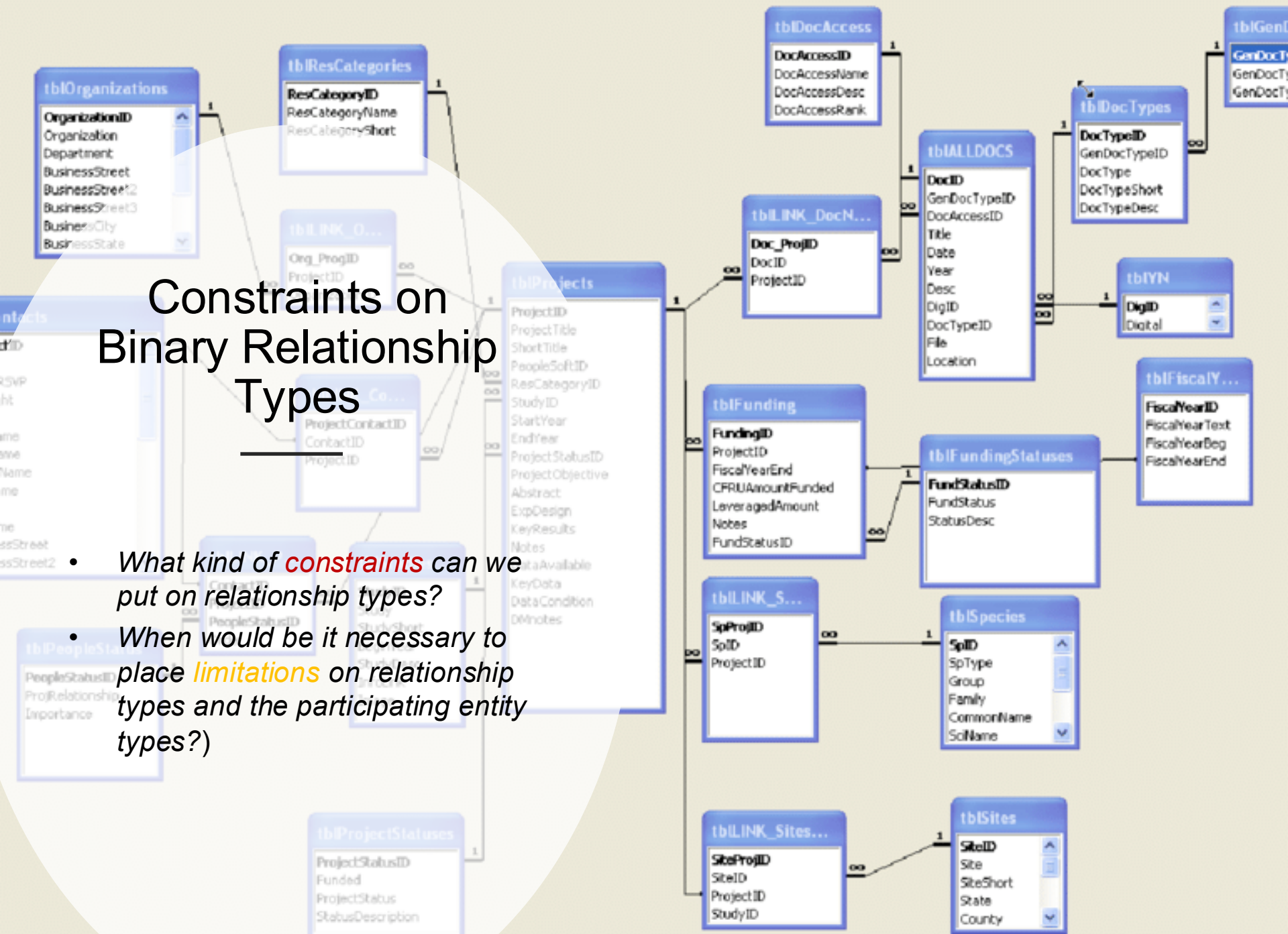


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

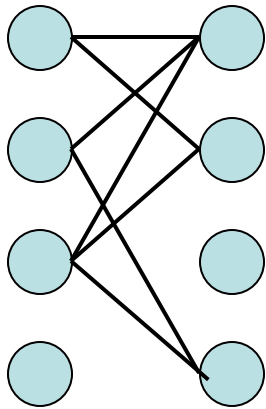
Constraints on Binary Relationship Types

- What kind of **constraints** can we put on relationship types?
- When would be it necessary to place **limitations** on relationship types and the participating entity types?)

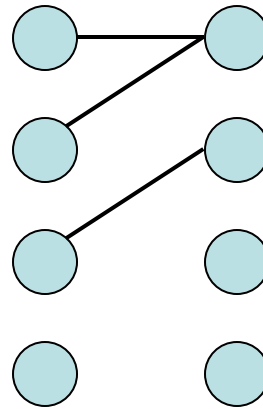


Cardinality Ratios for Binary Relationships

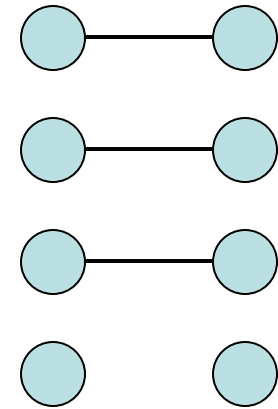
- Cardinality ratio: specifies the *maximum number* of relationship instances that an entity can participate in.



Many-many

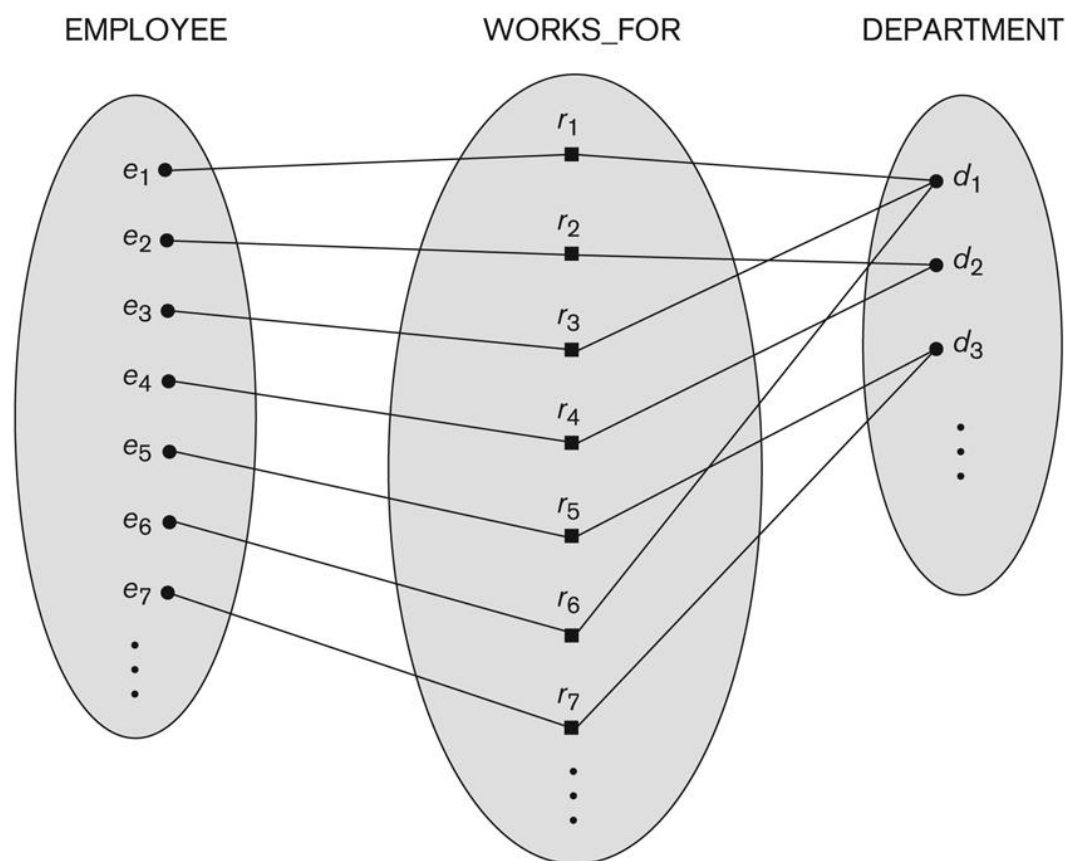


Many-one



One-one

1:N Relationship



Example: The WORKS_FOR binary relationship type

- DEPARTMENT:EMPLOYEE is of cardinality ratio **1:N**

Interpretation

- Each department can be related to (*employ*) **any** ($N = \text{zero or more}$) number of employees
- An employee can be related to (*work for*) **at most 1** department

Figure 3.9

Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

E-R Diagram Representation

Example: The WORKS_FOR binary relationship type

- Note the ER-diagram representation
- Are you able to find relationship types for the other two types of cardinality ratio? (1:1 and M:N)
- What are the implied **rules** that each of the relationship type represent?

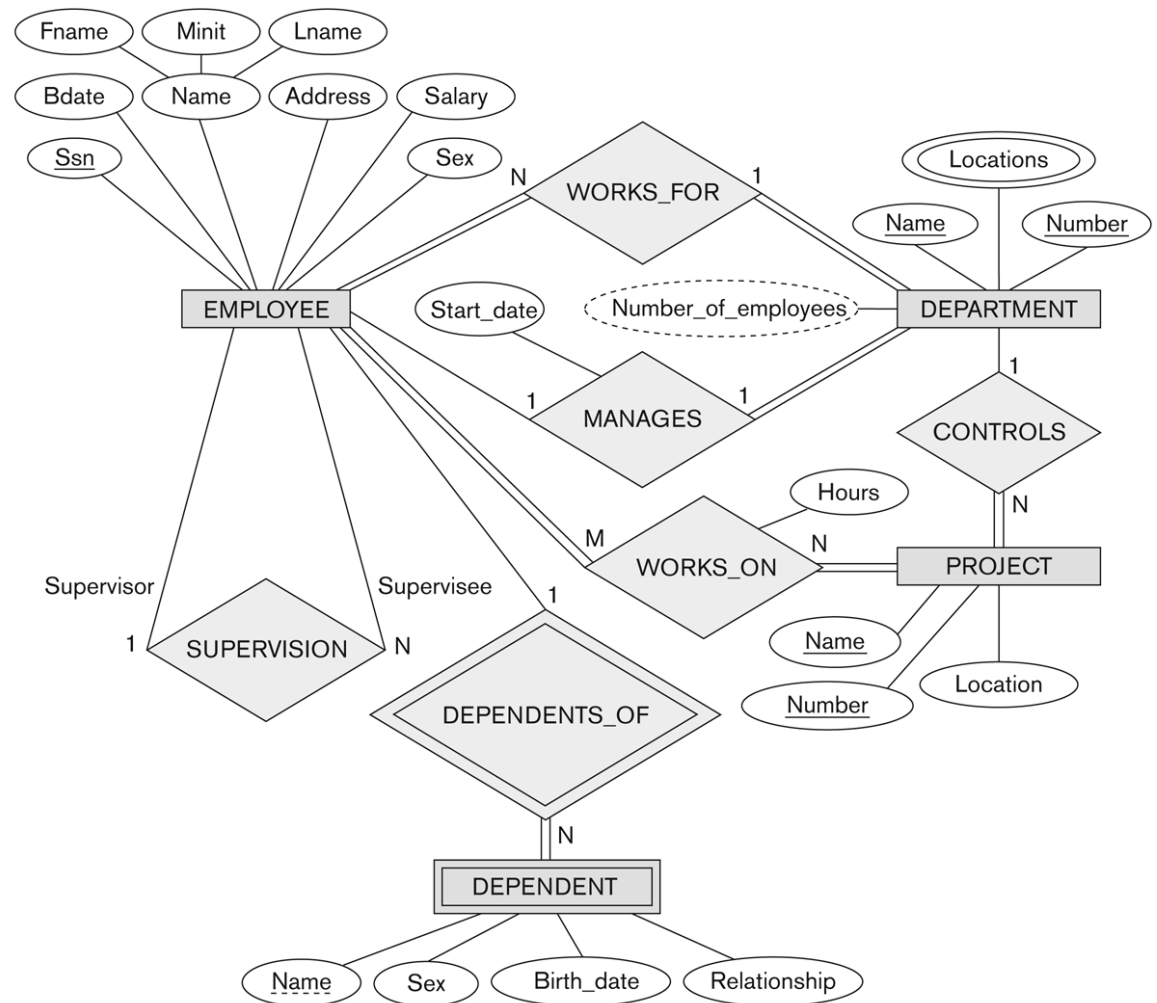
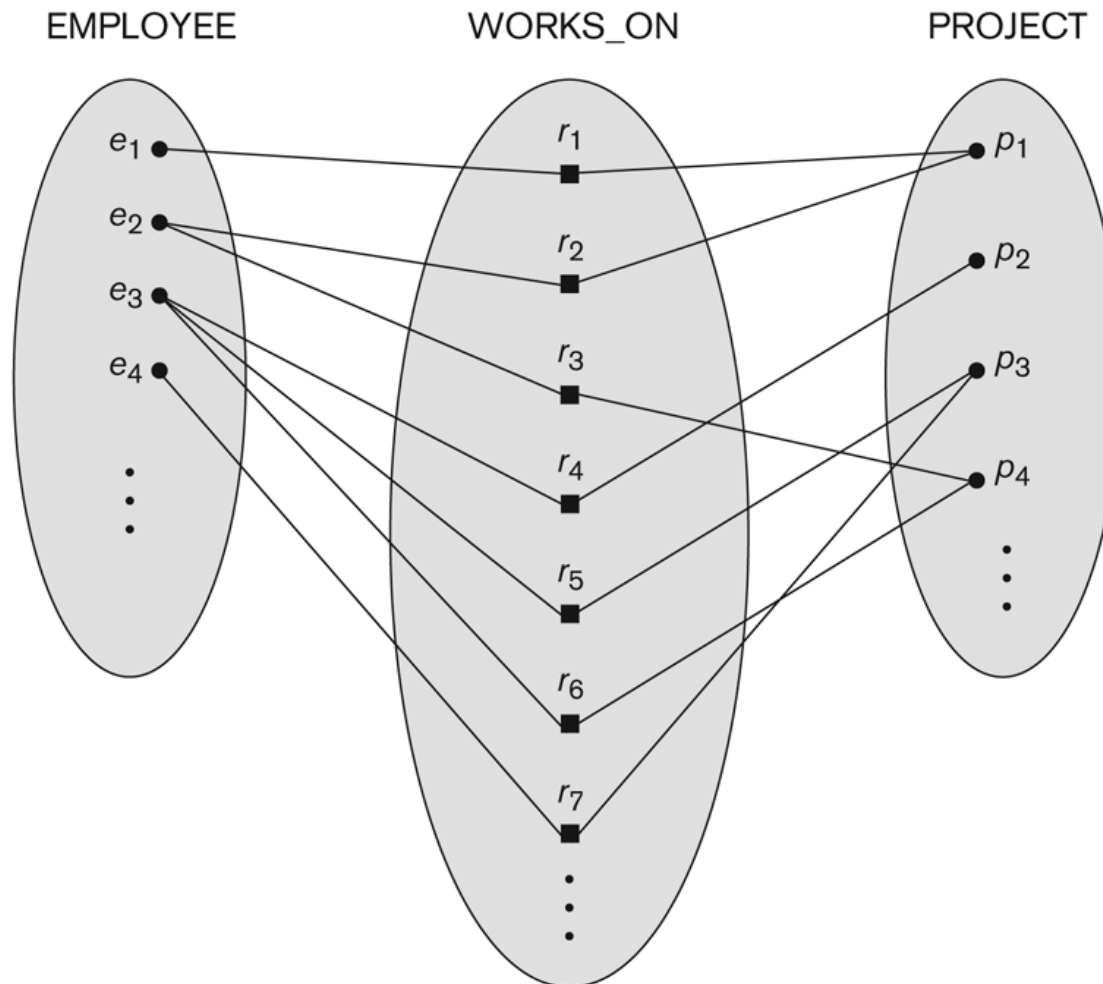


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Many-to-many (M:N) Relationship



Example: The WORKS_ON binary relationship type

- EMPLOYEE:PROJECT is of cardinality ratio **M:N**

Interpretation

- What is the implied *miniworld* rule/user requirement here?

Figure 3.13

An M:N relationship,
WORKS_ON.

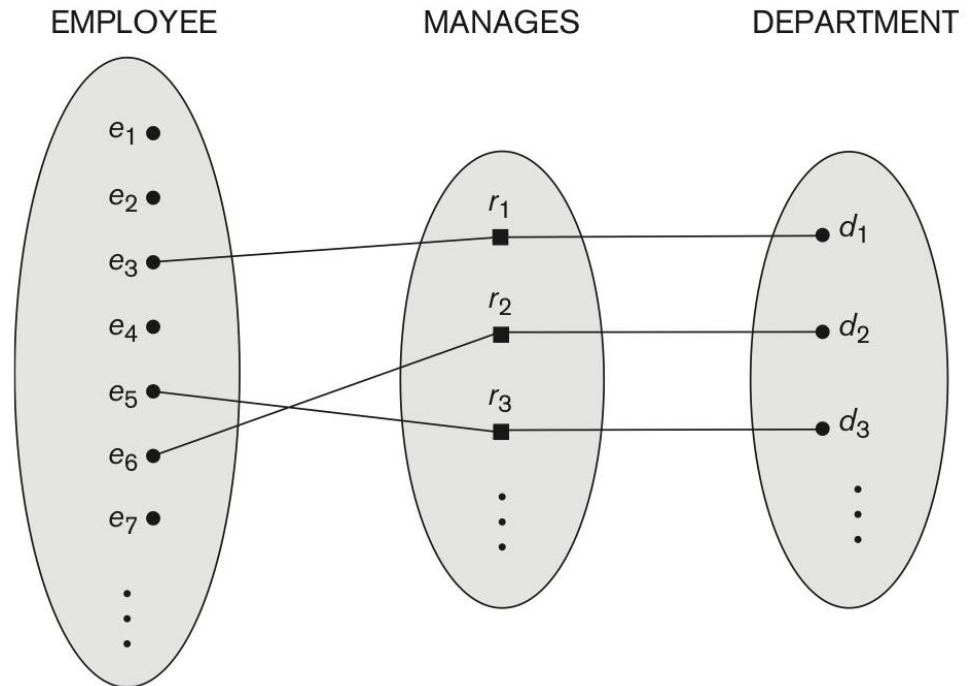
One-to-One (1:1) Relationship

Example: The MANAGES binary relationship type

- EMPLOYEE:DEPARTMENT is of cardinality ration **1:1**

Interpretation

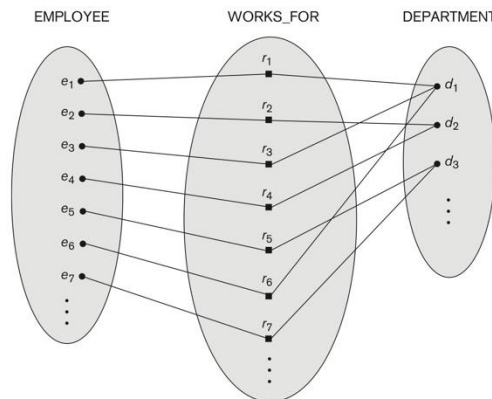
- What is the implied *miniworld* rule/user requirement here?
- *What's the maximum number of managers a department entity can have?*
- *Can an employee entity manage two different departments at the same time?*



Participation Constraints and Existence Dependencies

The **participation constraint** specifies whether the *existence* of an **entity** depends on its being related to another **entity** via the **relationship type**

- specifies the *minimum* number of relationship instances that each entity can participate in
- also called the **minimum cardinality constraint**
- **cardinality ratio constraint + participation constraint**
= **structural constraints** (of a relationship type)



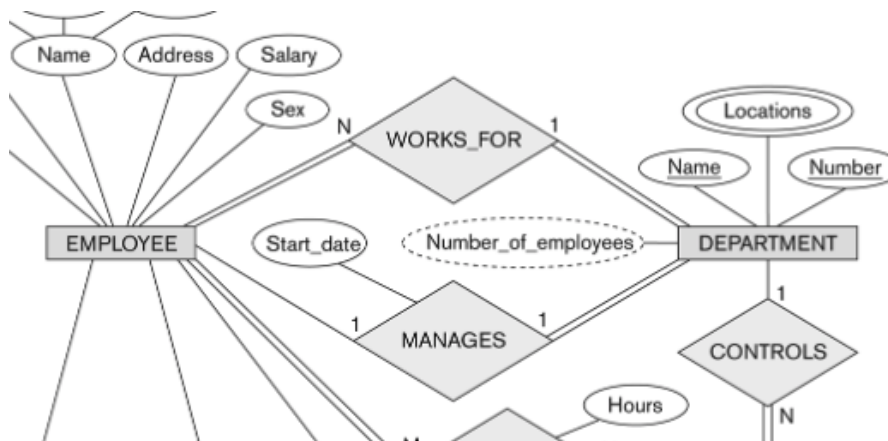
Example: The WORKS_FOR binary relationship type

- DEPARTMENT:EMPLOYEE is of cardinality ration **1:N**
- What is the **minimum** number of departments that an employee must work for? Can the value be zero?
- What is the **minimum** number of employees that a department has?
 - Can it be zero? Can an “**empty**” department be allowed to exist?
 - Does the participation constraints rely on **miniworld** assumptions?

Participation Constraints and Existence Dependencies

• Total Participation

- Every entity in the *total set* of the entity type must be related to the other entity type via the corresponding relationship
- Also called **existence dependency**
- Represented as **double line** in ER diagram



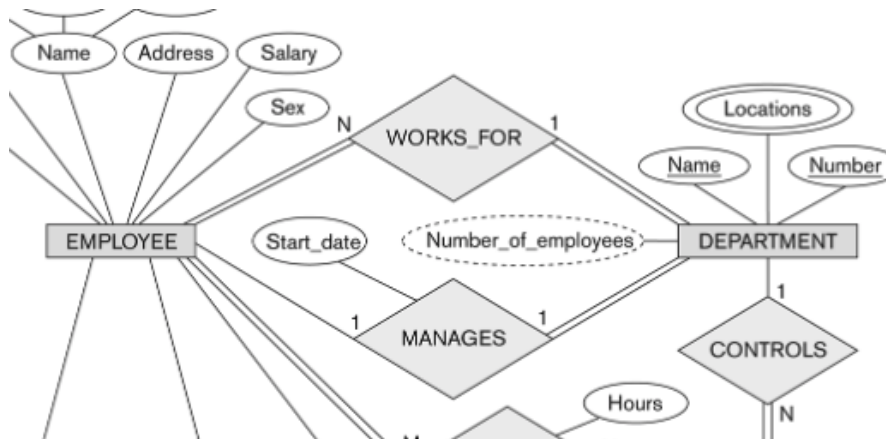
Example: The WORKS_FOR binary relationship type

- DEPARTMENT:EMPLOYEE is of cardinality ration **1:N**
- The Miniworld rule
 - An employee entity can only **exist** if it works for at least one department
 - → *every employee entity in the total set of EMPLOYEE entity type must be related to a department entity via WORKS_FOR*
- Would it make sense to have an employee who doesn't work for any department in the miniworld representation?

Participation Constraints and Existence Dependencies

• Partial Participation

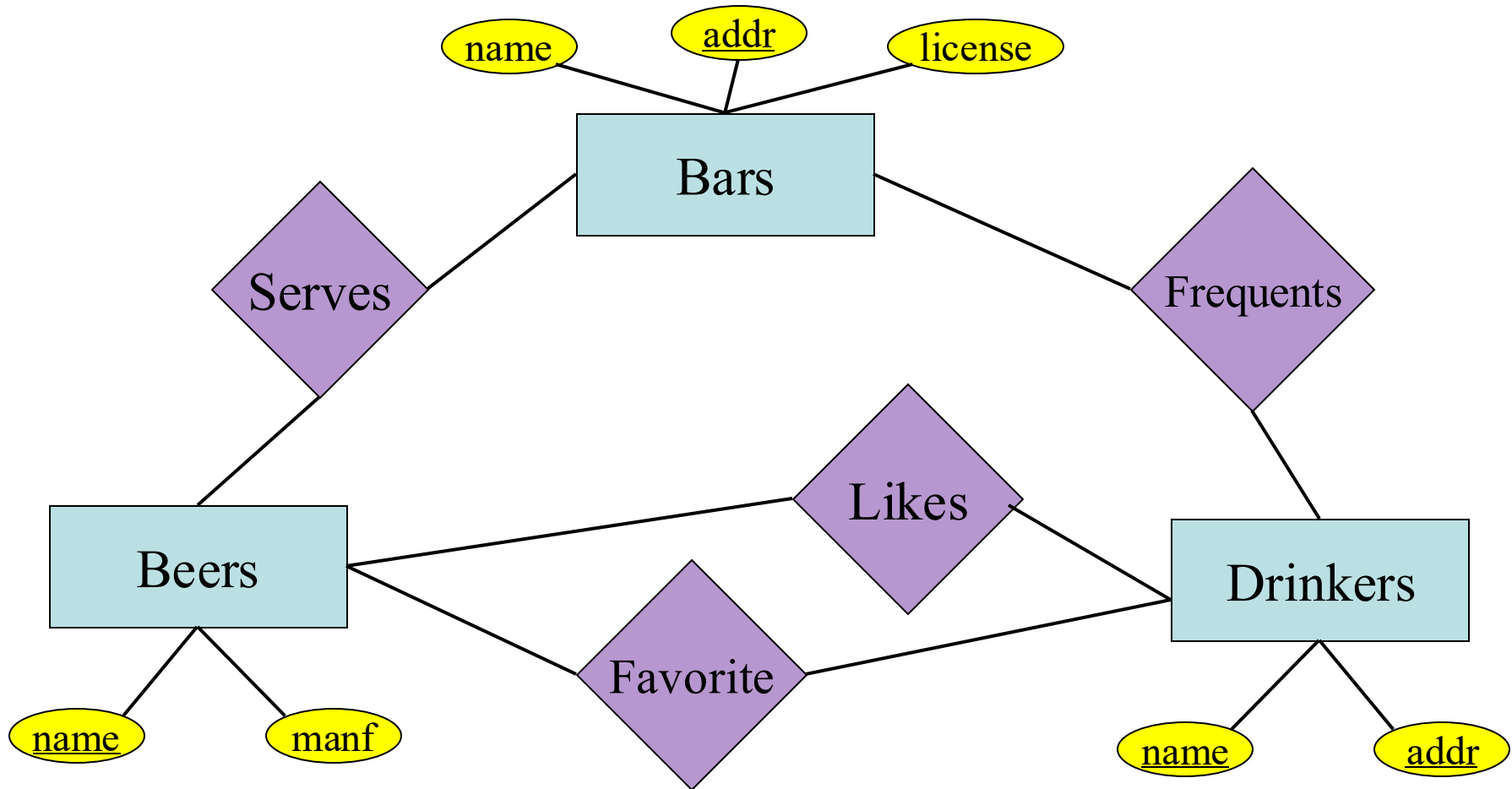
- **Some** or part of the set of entities in the entity set are related to another entity set, but not necessarily all
- Represented as **single line** in ER diagram



Example: The MANAGES binary relationship type

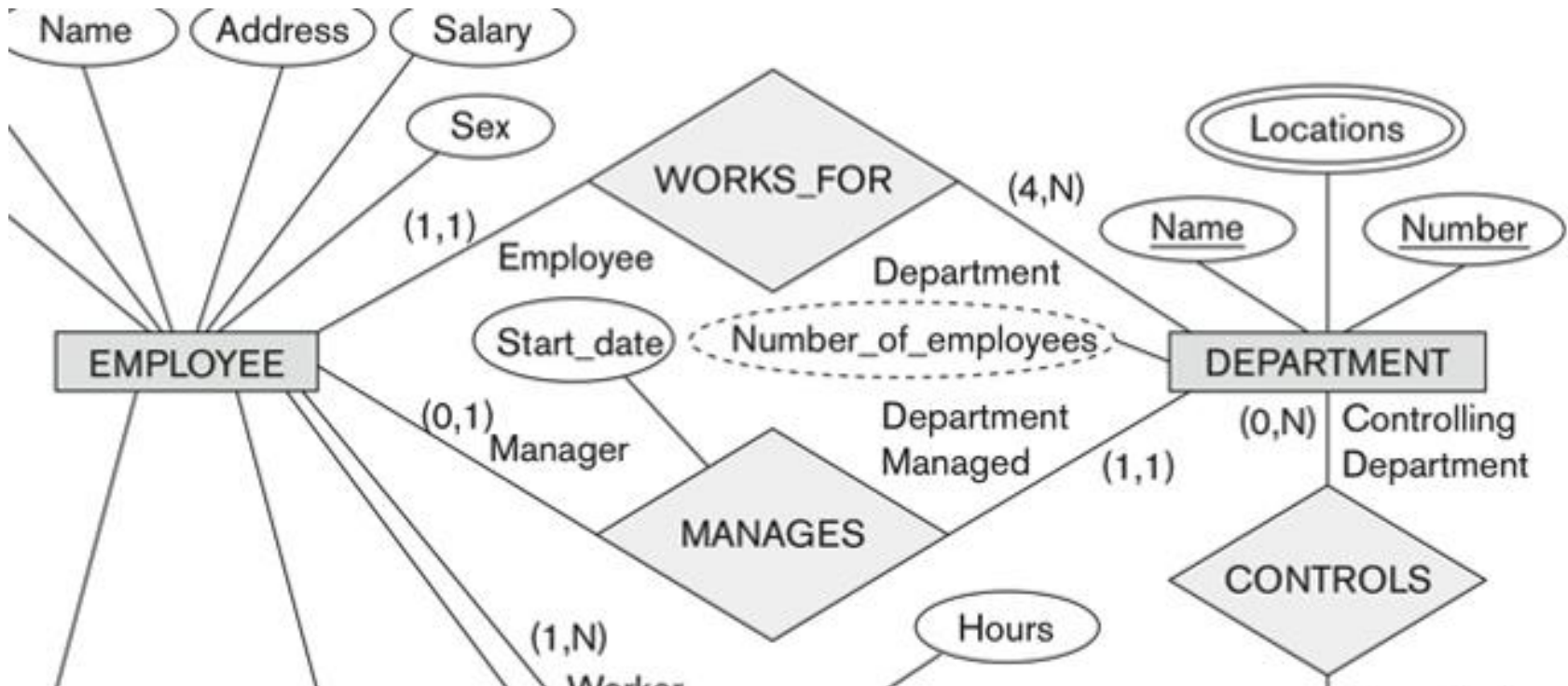
- DEPARTMENT:EMPLOYEE is of cardinality ratio **1:1**
- The Miniworld rule
 - We do not expect **every** employee to manage a department
 - → *only some employee entities are expected to participate in the MANAGES relationship*
- What about DEPARTMENT?
 - *Would a partial participation constraint make sense?*

Exercise



Alternative ER Notation for Structural Constraints on Relationships

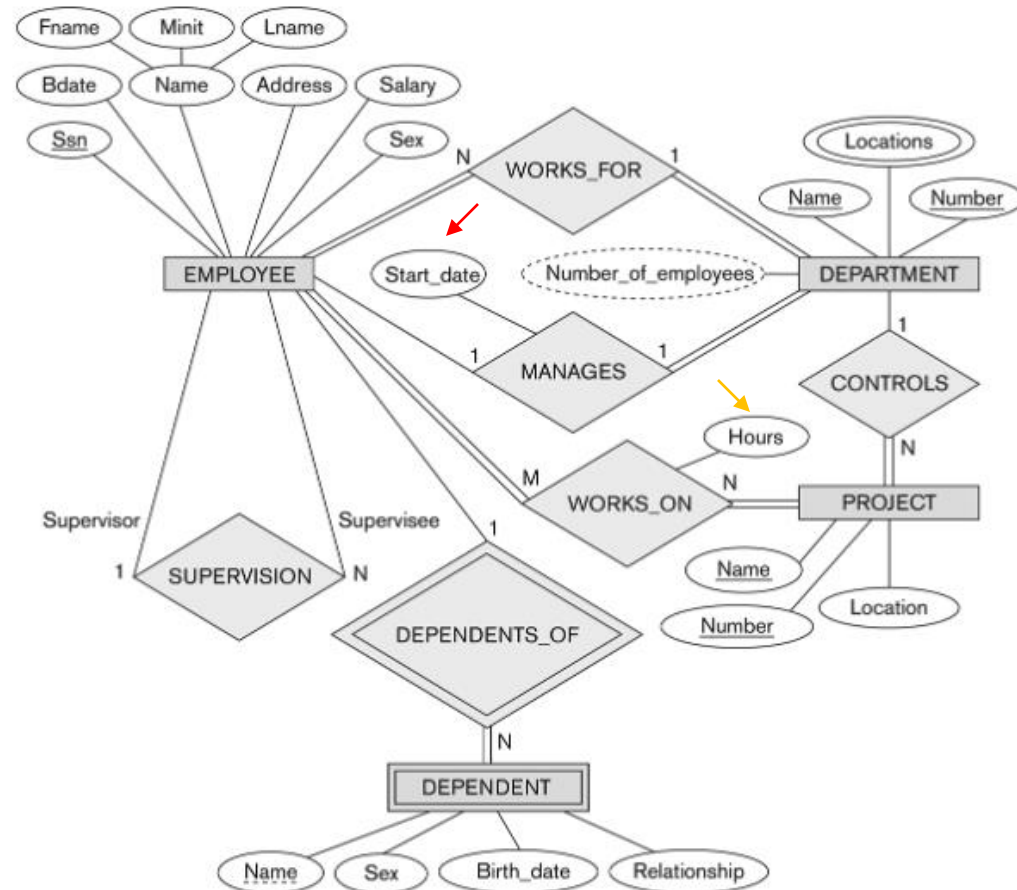
- The alternative notation $[(min, max)]$ combines the cardinality ratio and participation constraints into one
 - which one to choose is up to the database designer
- A pair of integer numbers (min, max) is to be associated with each participating entity type in a relationship (placed adjacent to it)
 - $0 \leq min \leq max$, and $max \geq 1$



For each entity e , e must participate in at least min and at most max relationship instances in R at any point in time.

Attributes of Relationship Types

Relationship types can also have attributes, similar to those of entity types

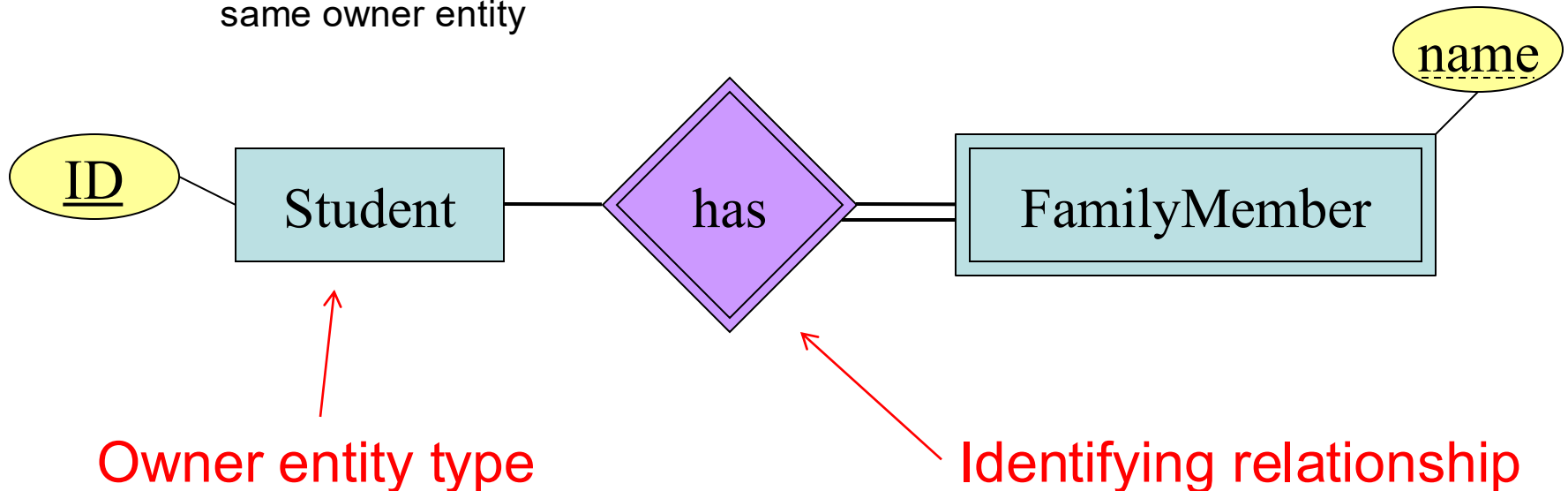


Weak Entity Types



Weak Entity Types

- **weak entity types:**
 - As opposed to *strong entity types*
 - Entity types that do not have key attributes of their own
- A weak entity cannot be *identified* without an **owner entity**
- A weak entity type normally has a **partial key**
 - The attribute that can uniquely identify weak entities that are related to the same owner entity



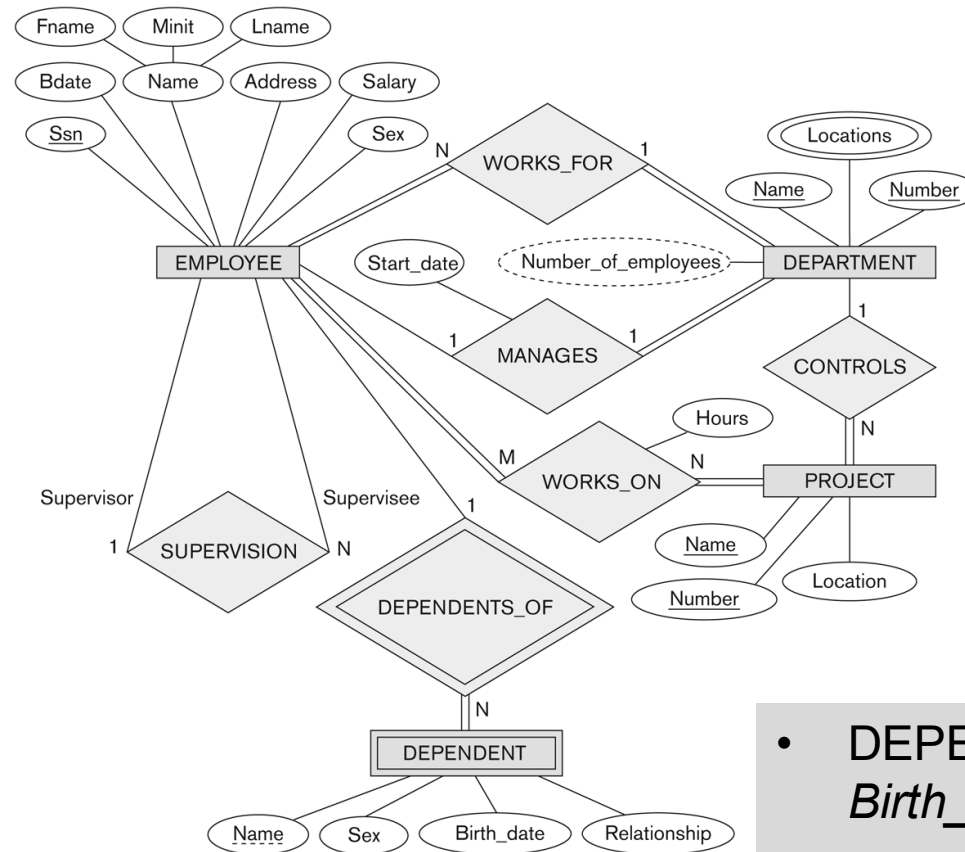


Figure 3.2

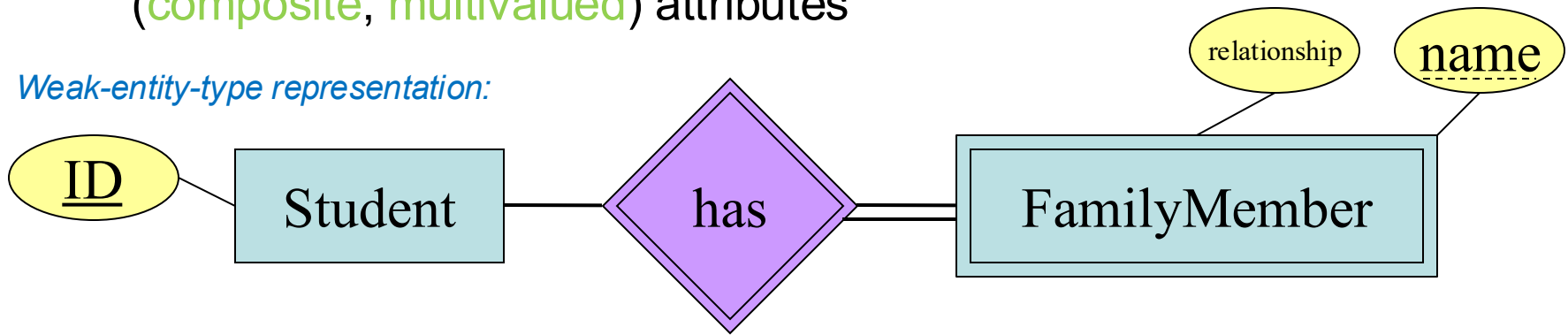
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

- **DEPENDENT**(*Name, Sex, Birth_date, Relationship*)
- Can you identify the owner entity type?
- What is the identifying relationship?
- What is the partial key?

Weak Entity Types

- Weak entity types can sometimes be represented as **complex** (**composite**, **multivalued**) attributes

Weak-entity-type representation:



Complex-attribute representation:

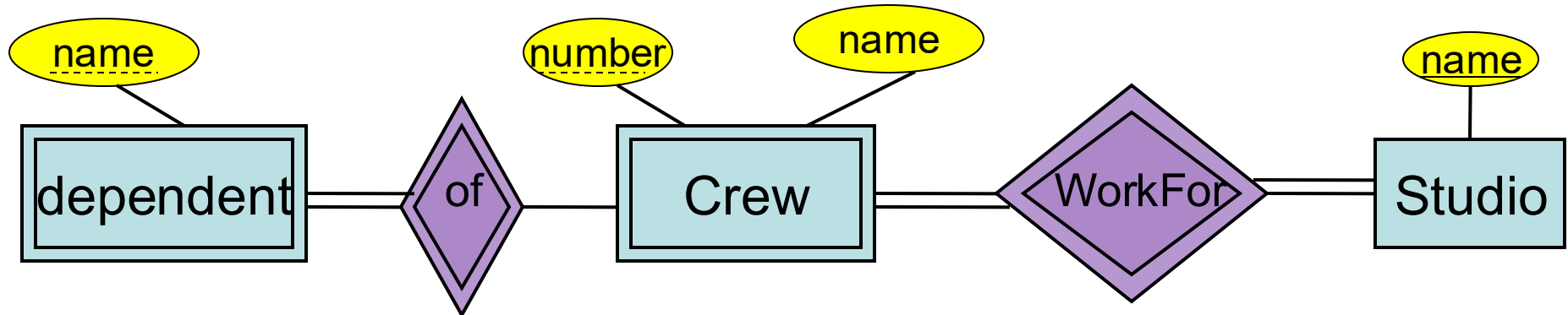


- The choice of which representation to use is made by the
? _____?

Chain of Weakness

In general, *any number of levels* of weak entity types can be defined

- An owner entity type may itself be a weak entity type

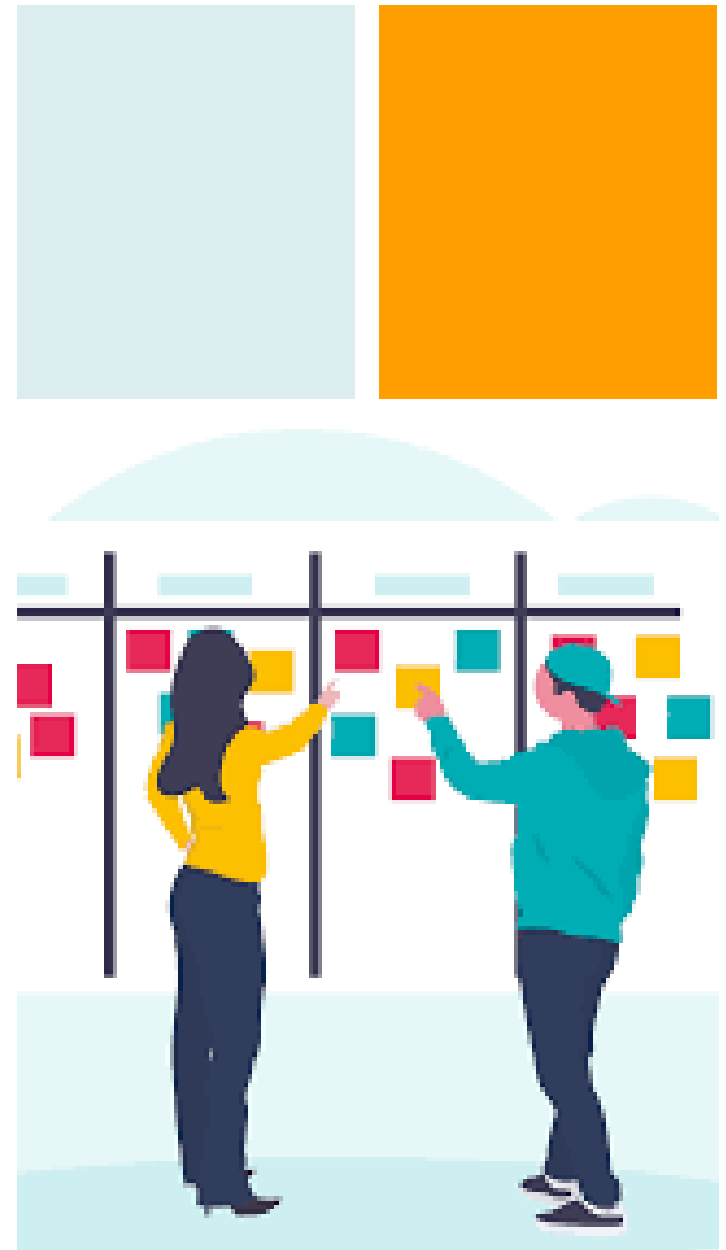


Design Choices for ER Conceptual Design

In general, the schema design process should be considered an **iterative refinement process**.

ER Modeling Guidelines (not conclusive):

1. A concept may be first modeled as an attribute and then **refined** into a relationship because it is determined that the attribute is a reference to another entity type.
2. An attribute that exists in several entity types may be elevated or **promoted** to an independent entity type.
3. An entity type related to another entity type may be **reduced** or demoted to an attribute of that entity type.

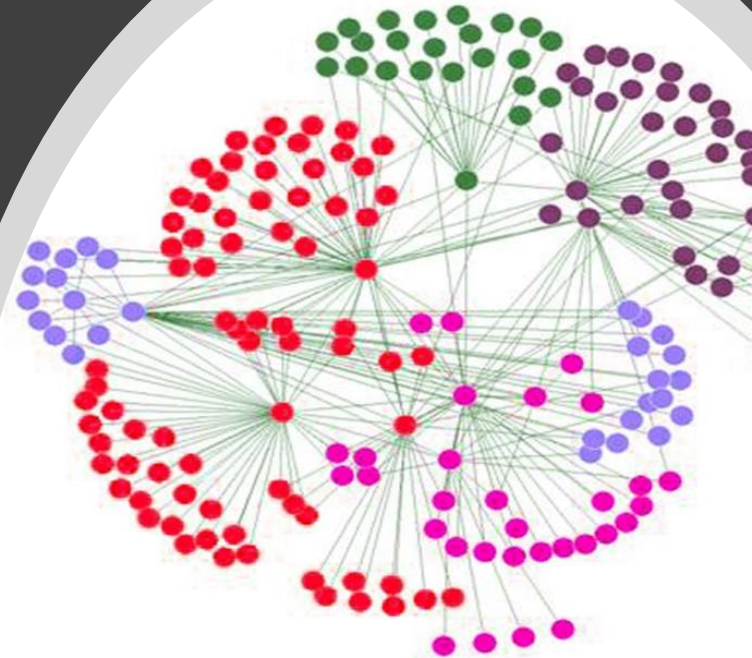


Relationship Types of Degree Higher than Two

- When to choose a higher-degree relationship design vs a binary one?
- How to specify constraints on higher-degree relationships?

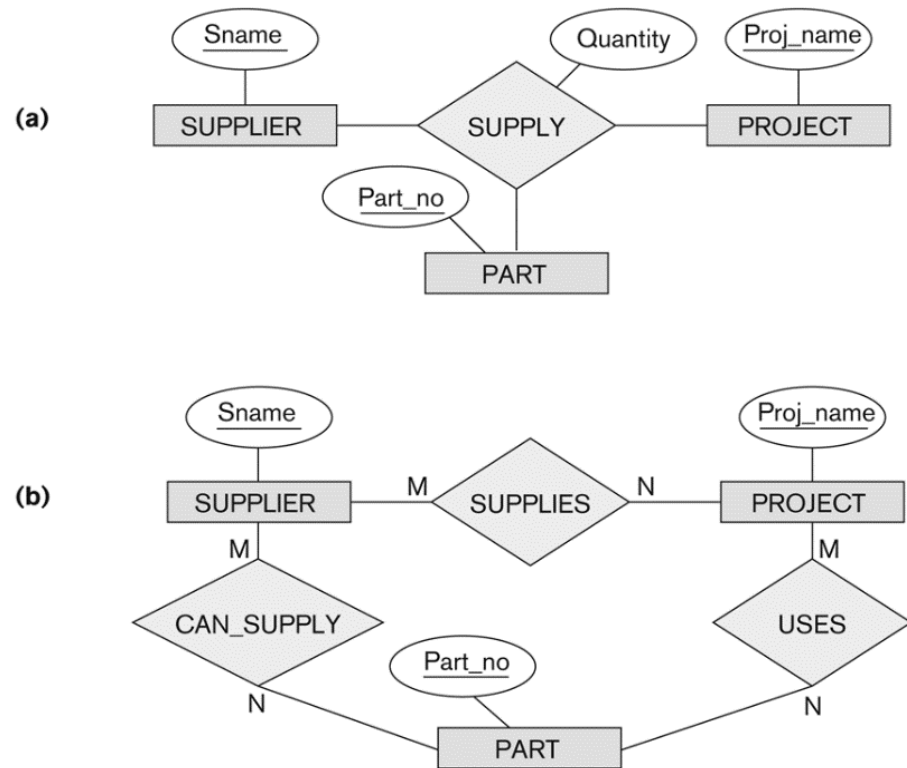


Oh...why
relationships are
so complex



Choosing between Binary and Higher-Degree Relationships

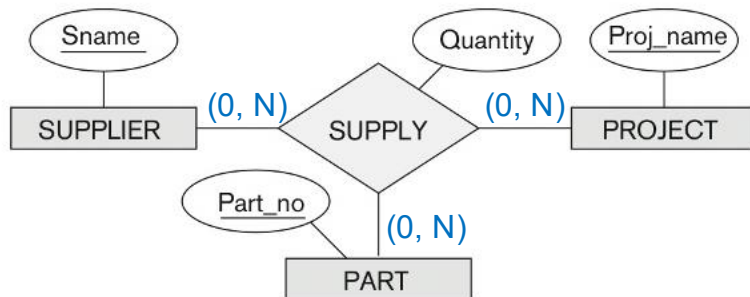
- It is often tricky to decide whether a particular relationship should be represented as a relationship type of degree n or should be broken down into several relationship types of smaller degrees.



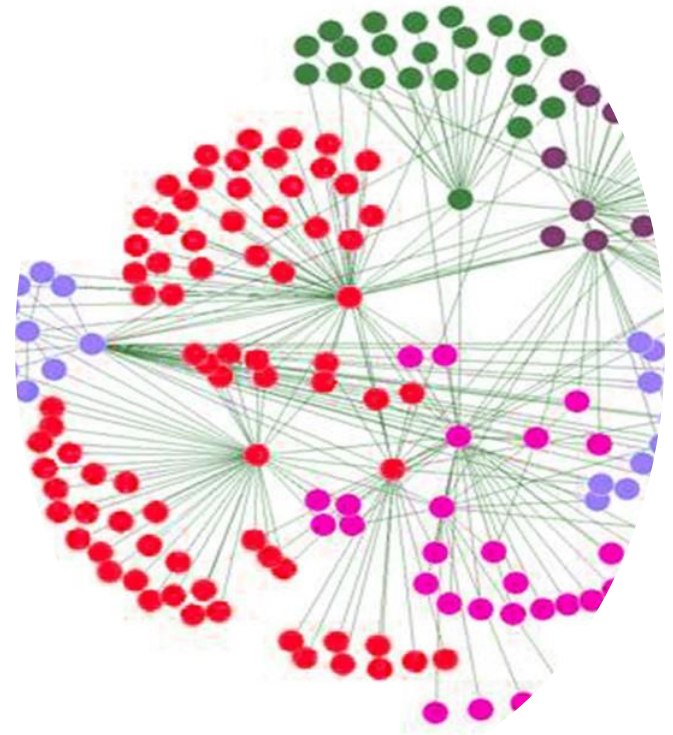
- Are the two relationship representations *equivalent* to each other?
- What is a proper *key* for the SUPPLY relationship in diagram (a)?
- Is it possible to migrate the 'Quantity' attribute of SUPPLY to one of the entity types?

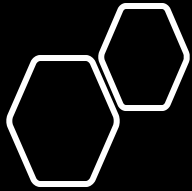
Constraints on Higher-Degree Relationships

- The primary constraint notation is based on the (min, max) notation to specify that each entity is related to at least **min** and at most **max** relationship instances in the relationship set



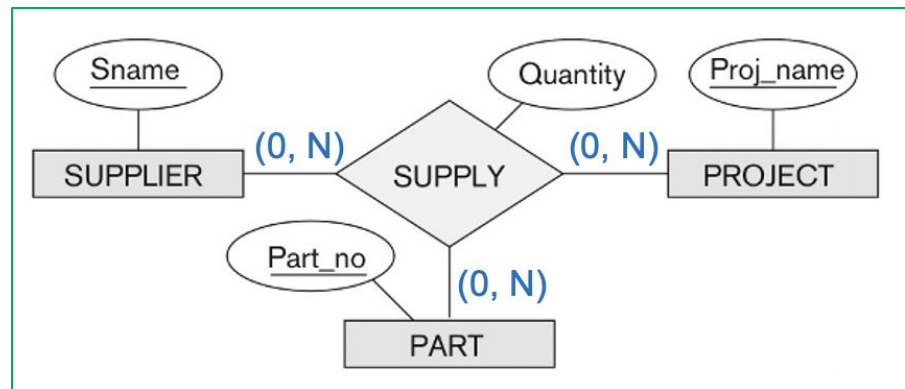
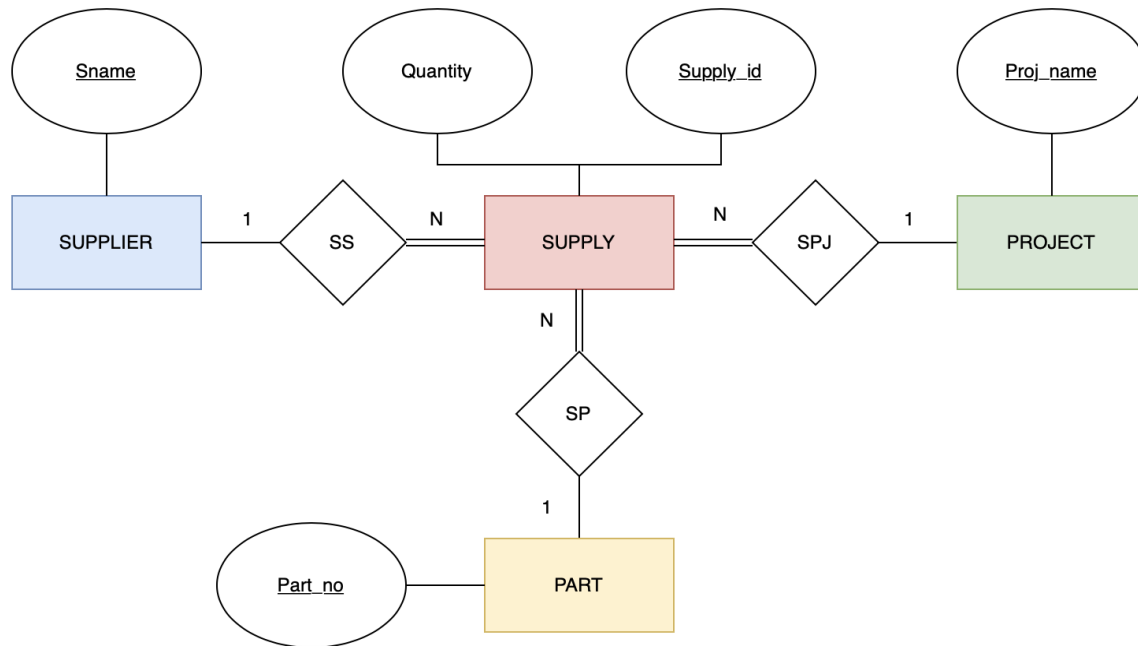
- Every supplier must supply a part to a project?
- Does every project require a part?
 - Yes/No
- Would all parts be used in some projects?
 - Yes/No





Choosing between Binary and Higher-Degree Relationships

- It is also possible to represent a relationship of higher degree as a **regular entity type**
- In this case, an artificial or **surrogate key** must be introduced along with the use of **binary 1:N relationships** to relate the new entity type to each of the participating entity types
- What about using a weak entity type instead? (**volunteer?**)



Example

Produce the ER-diagram:

Entities

- *Food* (*FoodID*, *Name*, *Type*)
- *Chef* (*ChefID*, *Name*, *Specialty*)
- *Nation* (*NationID*, *Name*, *CuisineType*)

Ternary Relationship

- *PreparesCuisine*(*Food*, *Chef*, *Nation*)

Constraints

- Not every chef prepares every food, but each chef can prepare more than one food.
- Each nation should have at least one food prepared.
- Each food must be prepared **at most** once.

Example

Use only binary relationships:

Entities

- *Food (FoodID, Name, Type)*
- *Chef (ChefID, Name, Specialty)*
- *Nation (NationID, Name, CuisineType)*

Ternary Relationship

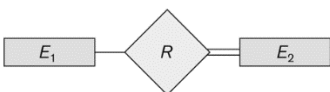
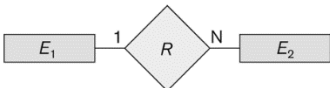
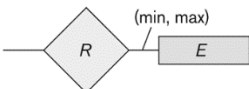
- *PreparesCuisine(Food, Chef, Nation)*

Constraints

- Not every chef prepares every food, but each chef can prepare more than one food.
- Each nation should have at least one food prepared.
- Each food must be prepared **at most** once.

Summary of notation for ER diagrams

Figure 3.14
Summary of the
notation for ER
diagrams.

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1: N for $E_1:E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

Summary of Lecture 3 Data Modeling Using the ER Model

- **What we learned?**
 - ER Model
 - Entity Types and Attributes
 - Key Attributes
 - Relationship Types
 - Structural Constraints of Relationships
 - Weak Entity Types
 - Ternary and Higher-Degree Relationships
- **What you understood the most?**
- **What are the most difficult to you?**

